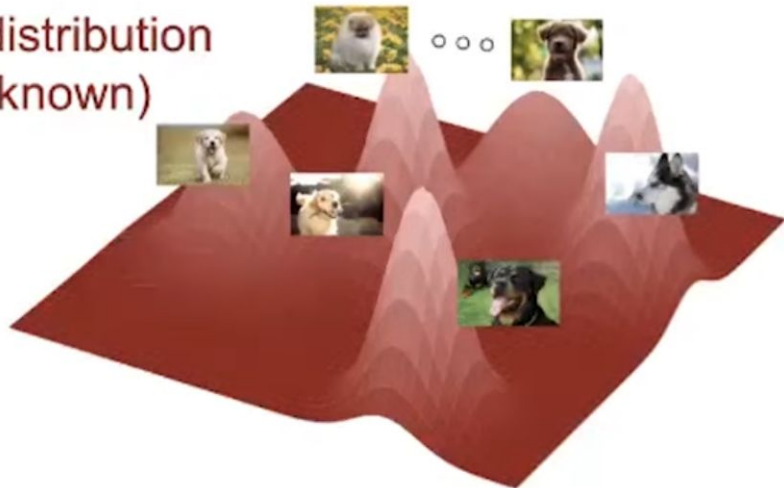


Generative Models

**EPFL
VITA**

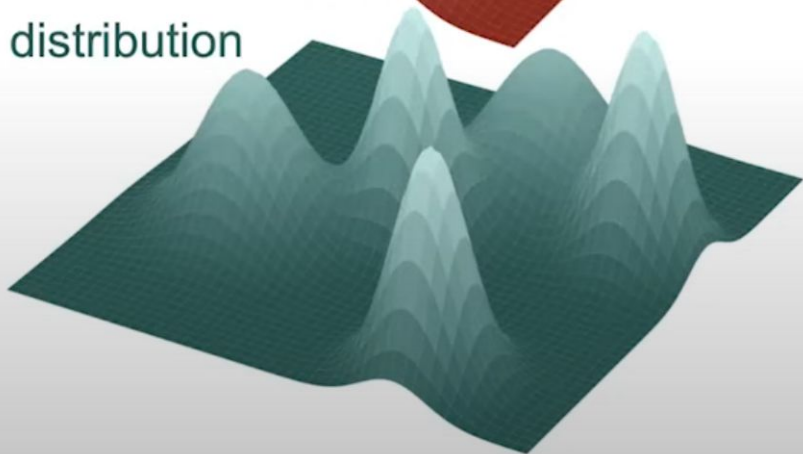
Data distribution
(unknown)

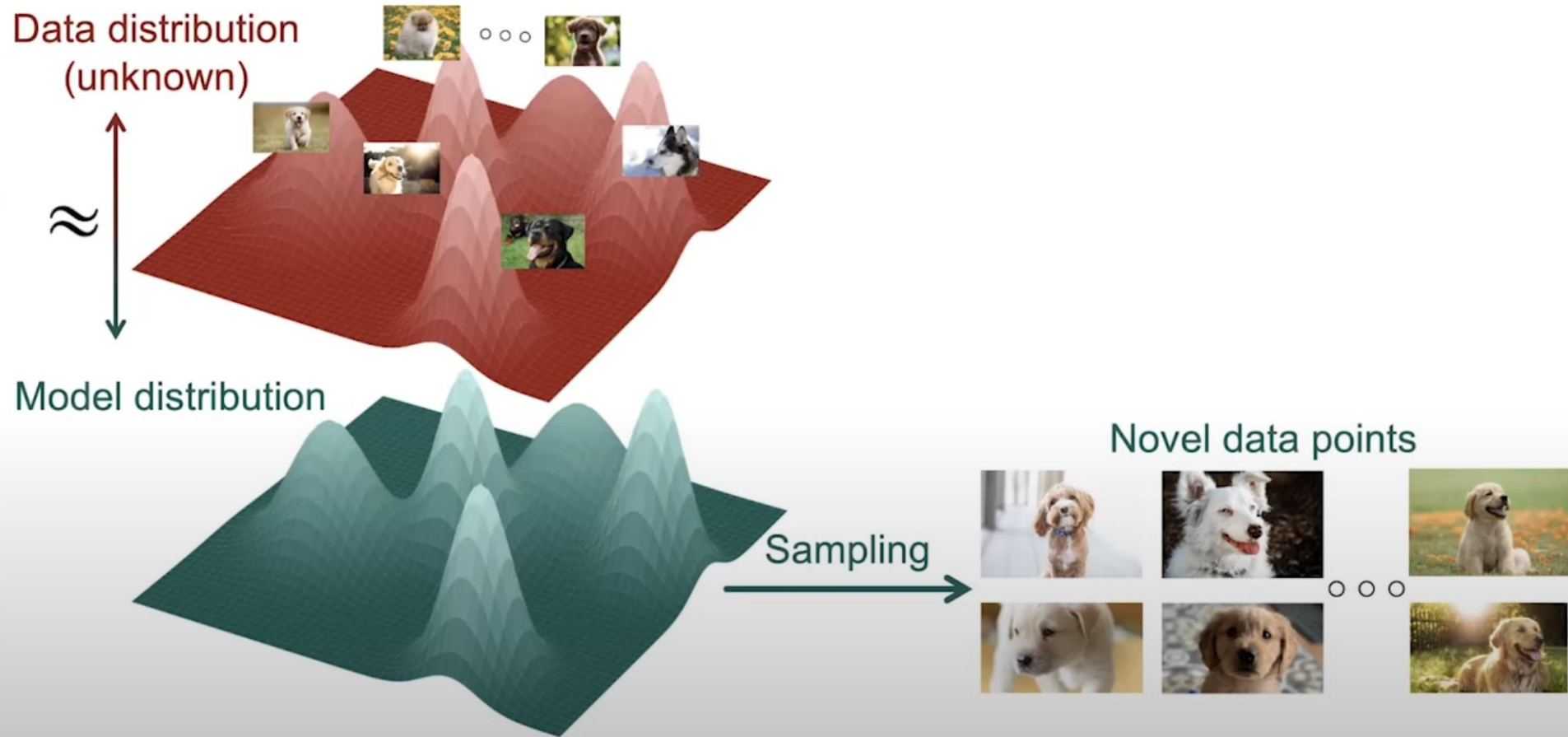


Data distribution
(unknown)

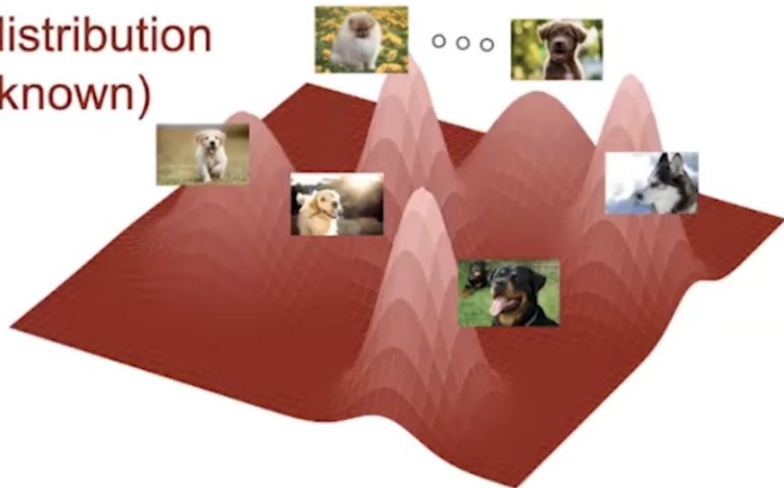


Model distribution





Data distribution
(unknown)



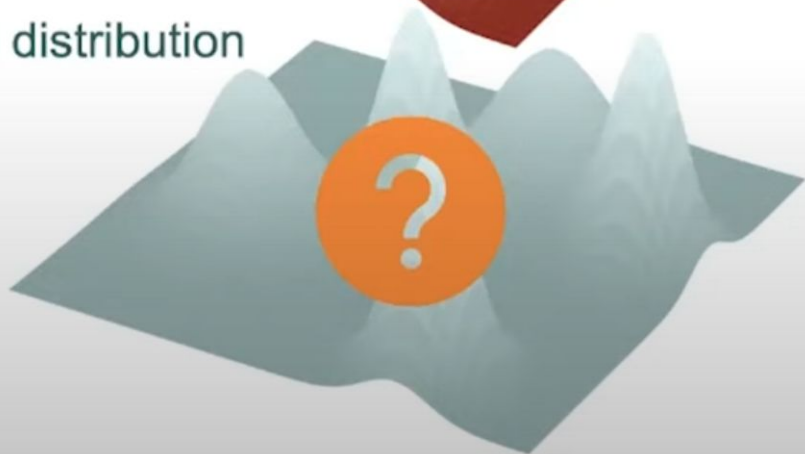
Data distribution is
extremely complex for
high dimensional data.

Data distribution
(unknown)



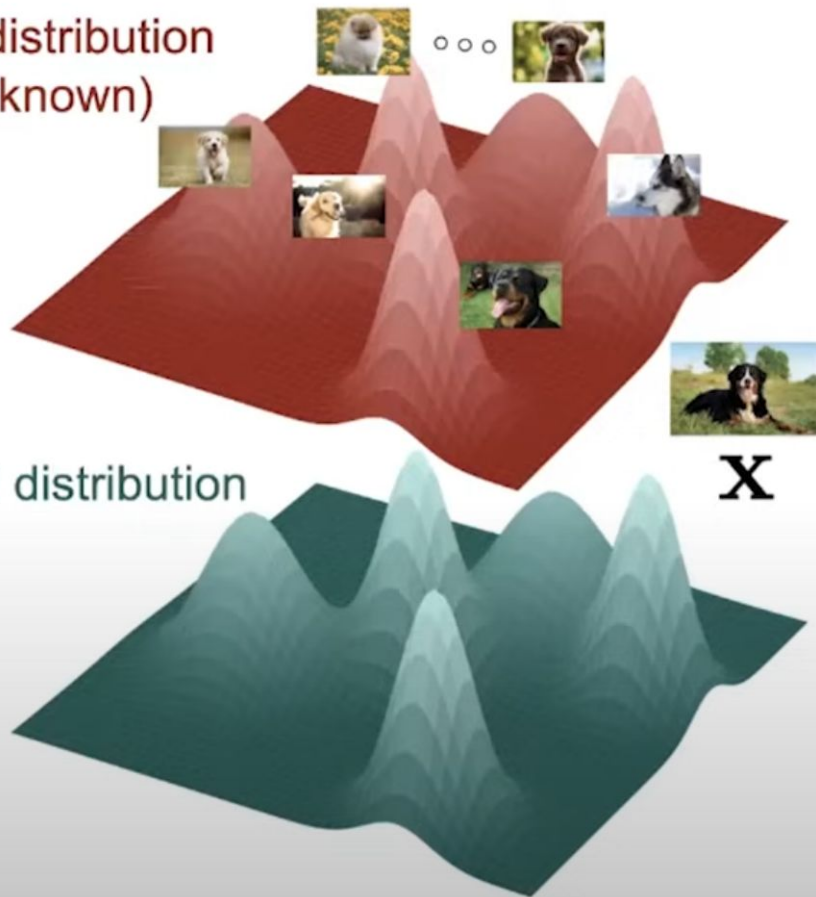
Data distribution is
extremely complex for
high dimensional data.

Model distribution

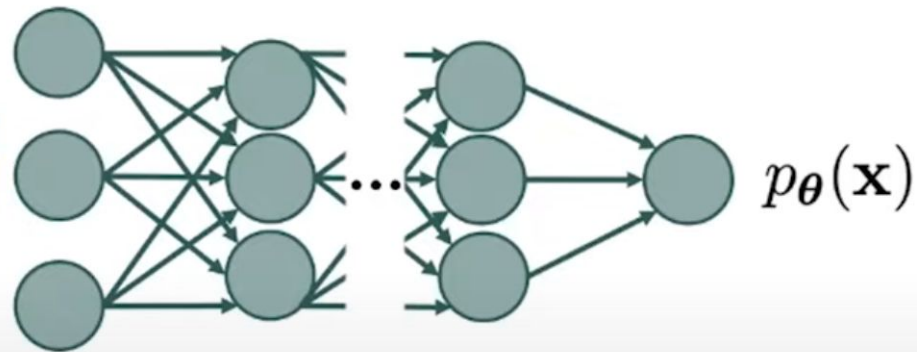


How to build a complex
model to fit the data
distribution?

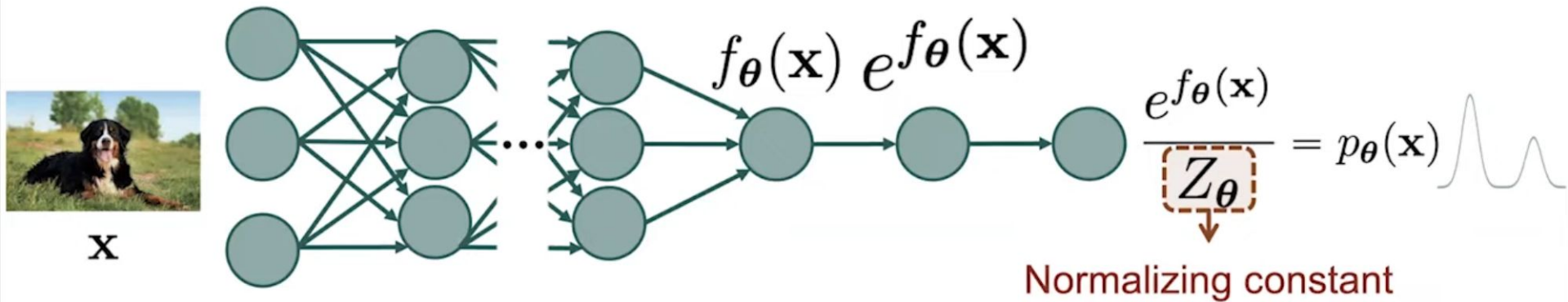
Data distribution
(unknown)



But it must be probability distribution !



Deep Neural Network



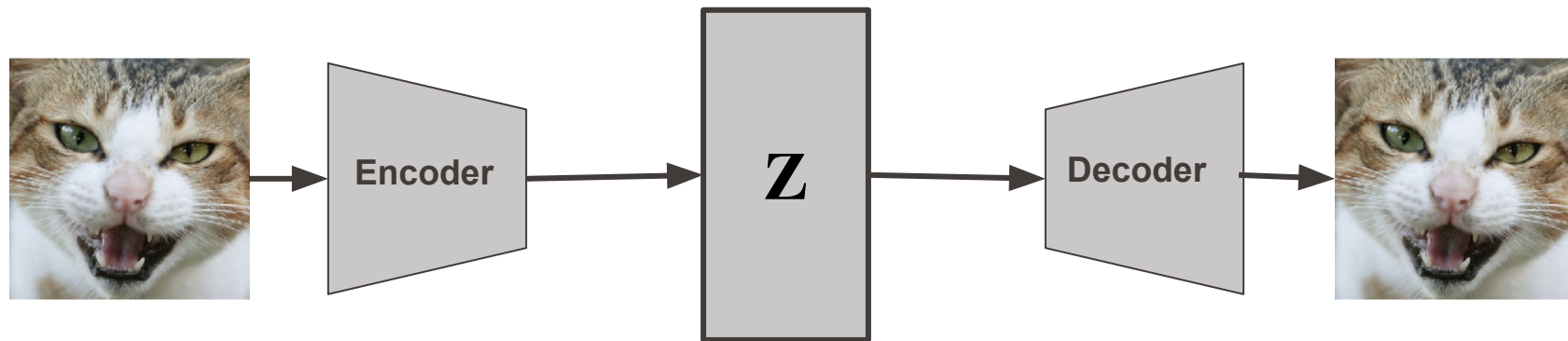
$$Z_{\theta} = \int \dots \int e^{-\beta E(\mathbf{x}, \mathbf{y})} d\mathbf{x} d\mathbf{y}$$

VAE

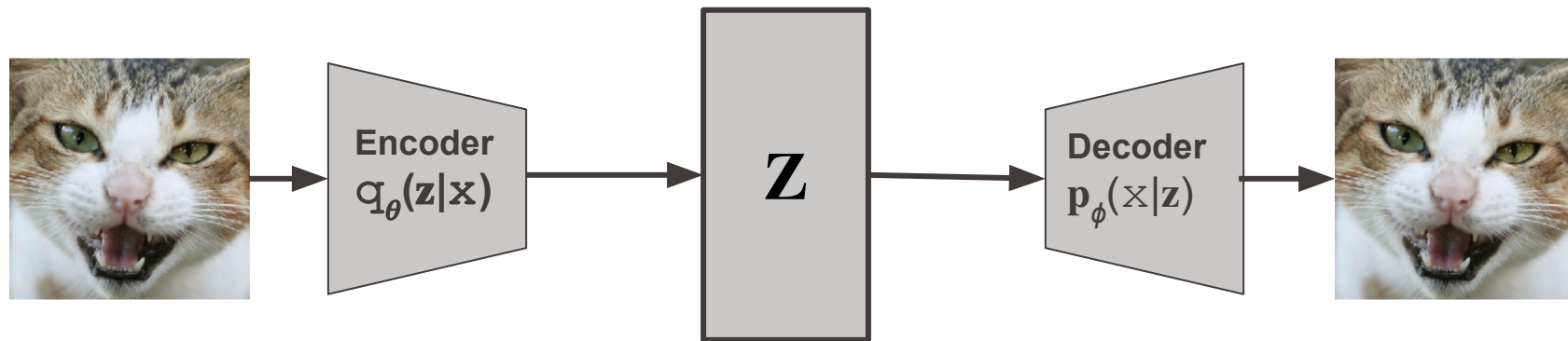
Flow-Matching

Score-Matching

Variational AutoEncoder (VAE)



How to sample?

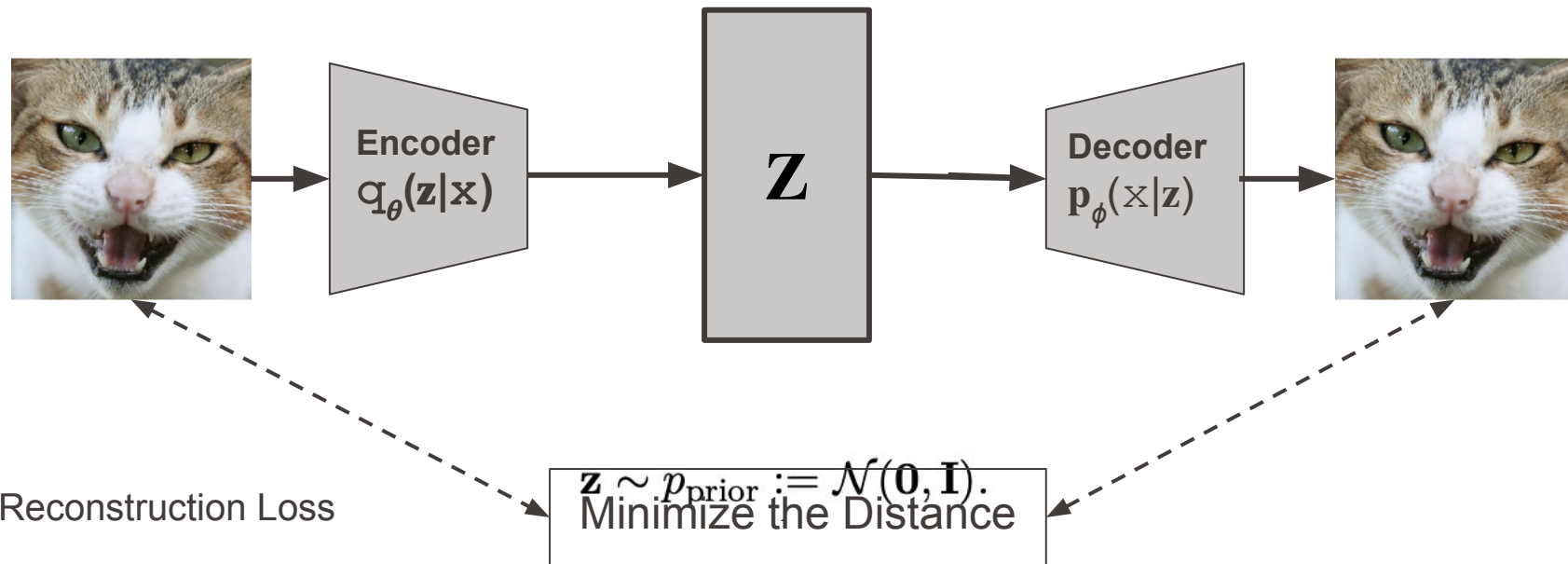


For any data point $\mathbf{x}$, the log-likelihood satisfies:

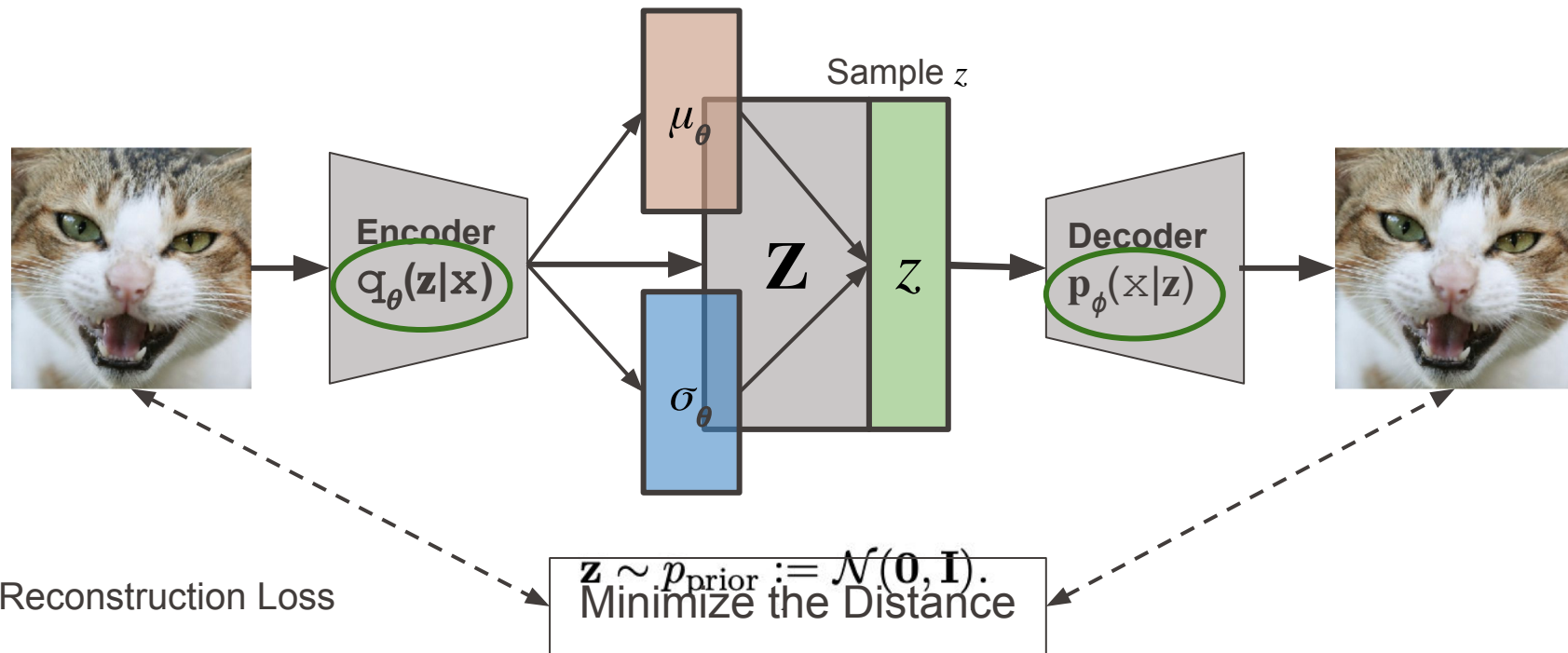
$$\log p_{\phi}(\mathbf{x}) \geq \mathcal{L}_{\text{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}),$$

where the ELBO is given by:

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z}|\mathbf{x})} [\log p_{\phi}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Term}} - \underbrace{\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{Latent Regularization}}.$$



Variational AutoEncoder



$$q_{\theta}(\mathbf{z}|\mathbf{x}) := \mathcal{N}(\mathbf{z}; \mu_{\theta}(\mathbf{x}), \text{diag}(\sigma_{\theta}^2(\mathbf{x}))),$$

$$p_{\phi}(\mathbf{x}|\mathbf{z}) := \mathcal{N}(\mathbf{x}; \mu_{\phi}(\mathbf{z}), \sigma^2 \mathbf{I}),$$

```
# 1. ENCODE: Get the mean and log-variance  
mu, logvar = model.encode(x)
```

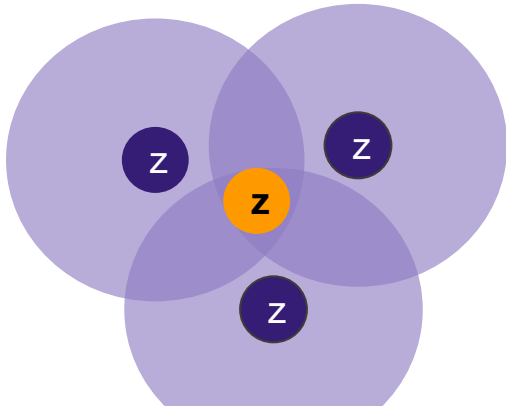
```
# 2. SAMPLE (Reparameterize): Get z  
std = torch.exp(0.5 * logvar)  
eps = torch.randn_like(std)  
z = mu + eps * std
```

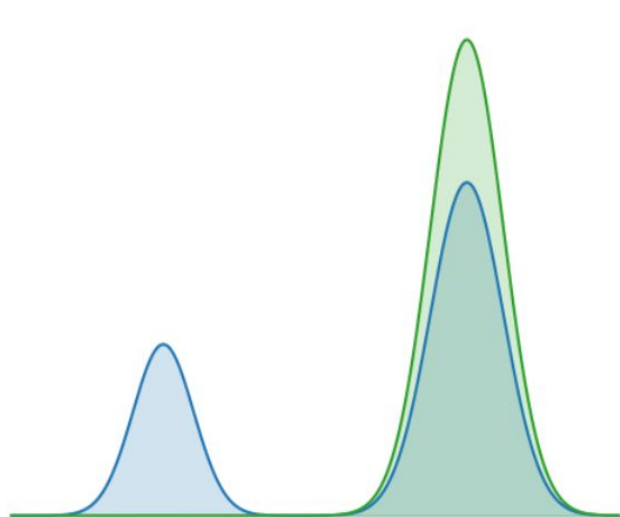
} Reparametrization Trick

```
# 3. DECODE: Reconstruct the image  
x_pred = model.decode(z)
```

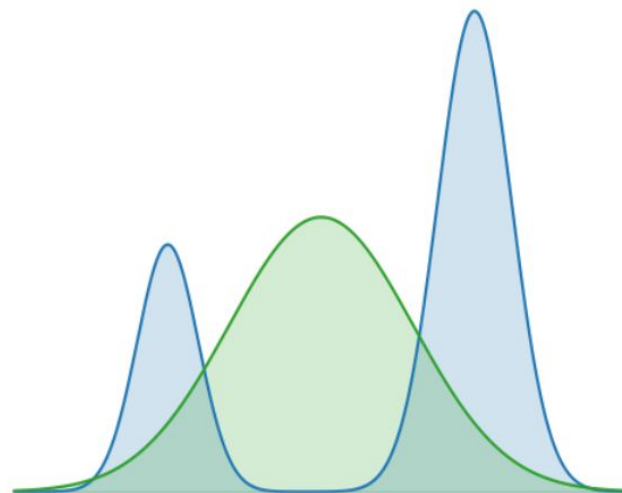
```
# --- THE LOSS ---
```

```
recon_loss = F.mse_loss(x_pred, x, reduction='sum')  
kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())  
total_loss = recon_loss + kl_loss
```



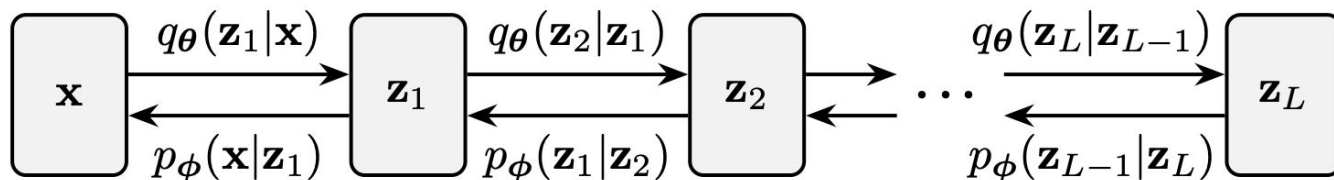


mode-seeking

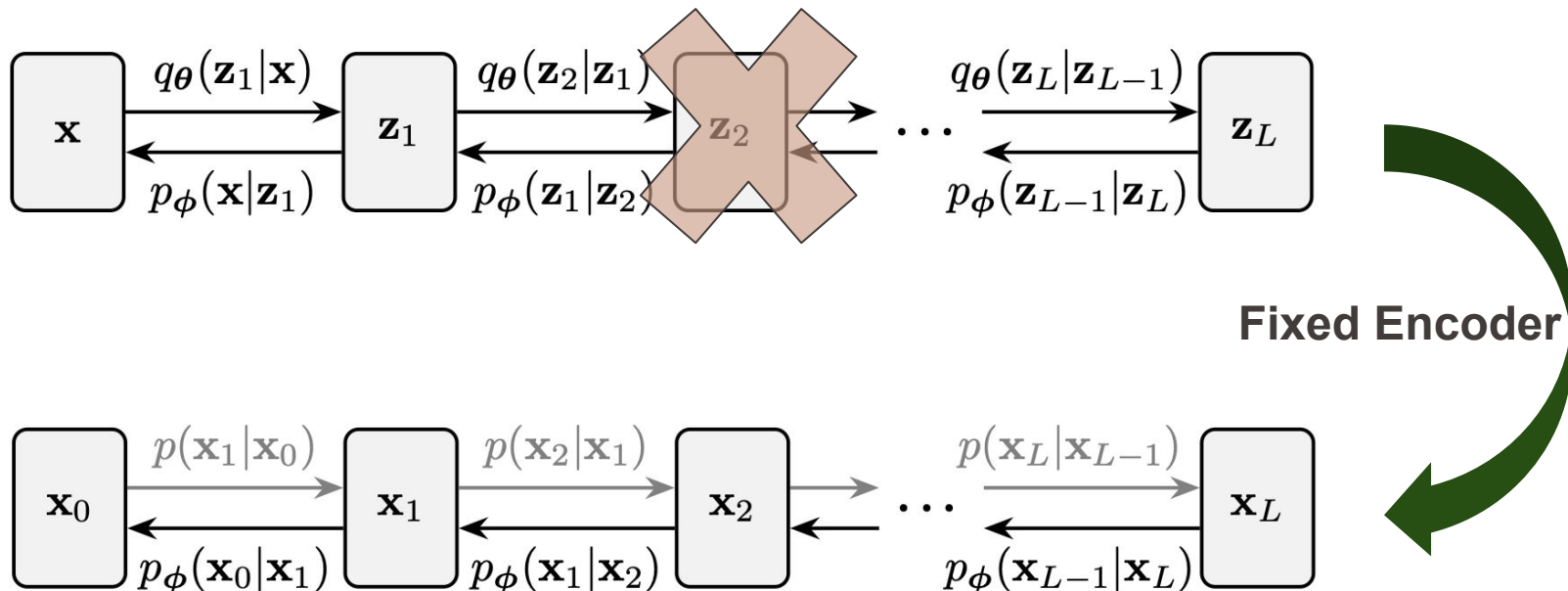
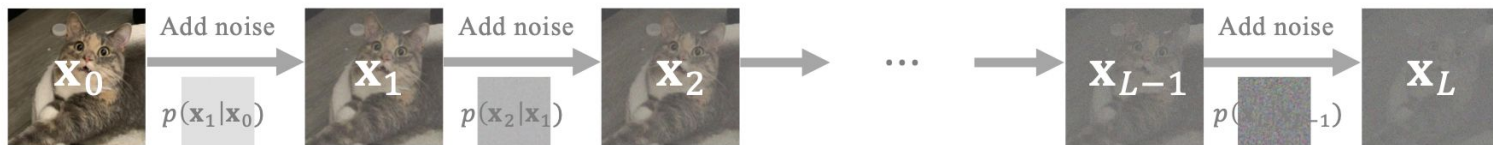
GANs

mode-covering

Likelihood-based (VAE, ...)



- **Problem:** Their training poses a unique challenge, because encoder and decoder are optimized jointly, learning becomes unstable. Gradient information is often weak in deeper layers making it difficult for them to contribute meaningfully.

 $\mathbf{x}_0 \sim p_{\text{data}}$ 

Data
DistributionReverse
DiffusionNoise
Distribution

$$p(\mathbf{x}_0) \simeq q(\mathbf{x}_0)$$



$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t))$$



Learned drift and covariance functions

$$\mathbf{x}_0 \sim p_{\text{data}}$$



Denoise

$$p(\mathbf{x}_0|\mathbf{x}_1)$$



Denoise

$$p(\mathbf{x}_1|\mathbf{x}_2)$$



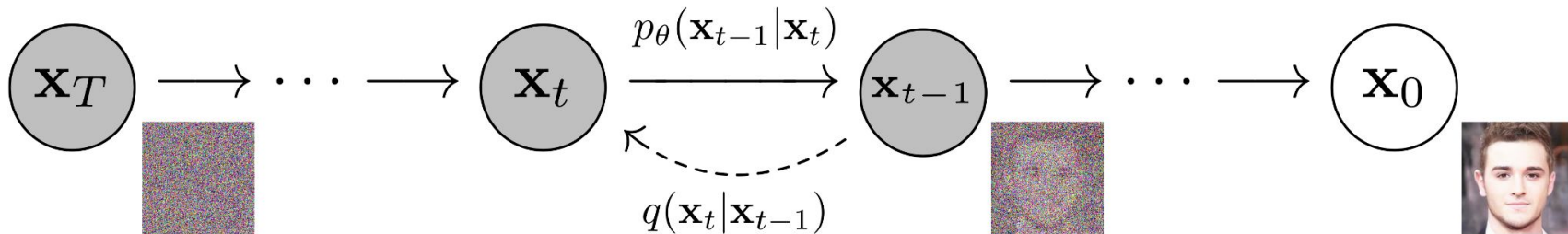
...



Denoise

$$p(\mathbf{x}_{L-1}|\mathbf{x}_L)$$





Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
$$\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
 - 6: **until** converged
-

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i) = p(\mathbf{x}_i|\mathbf{x}_{i-1}) \underbrace{\frac{p_{i-1}(\mathbf{x}_{i-1})}{p_i(\mathbf{x}_i)}}_{\text{intractable}}. \quad \Rightarrow \quad p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = p(\mathbf{x}_i|\mathbf{x}_{i-1}) \frac{p(\mathbf{x}_{i-1}|\mathbf{x})}{p(\mathbf{x}_i|\mathbf{x})}.$$

The following equality holds:

$$\begin{aligned} & \mathbb{E}_{p_i(\mathbf{x}_i)} [\mathcal{D}_{\text{KL}}(p(\mathbf{x}_{i-1}|\mathbf{x}_i) \| p_\phi(\mathbf{x}_{i-1}|\mathbf{x}_i))] \\ &= \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p(\mathbf{x}_i|\mathbf{x})} [\mathcal{D}_{\text{KL}}(p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) \| p_\phi(\mathbf{x}_{i-1}|\mathbf{x}_i))] + C, \end{aligned}$$

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = \mathcal{N} \left(\mathbf{x}_{i-1}; \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}, i), \sigma^2(i)\mathbf{I} \right),$$

$$p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}) = \mathcal{N} \left(\mathbf{x}_{i-1}; \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}, i), \sigma^2(i)\mathbf{I} \right),$$

where

$$\boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}, i) := \frac{\bar{\alpha}_{i-1}\beta_i^2}{1 - \bar{\alpha}_i^2}\mathbf{x} + \frac{(1 - \bar{\alpha}_{i-1}^2)\alpha_i}{1 - \bar{\alpha}_i^2}\mathbf{x}_i, \quad \sigma^2(i) := \frac{1 - \bar{\alpha}_{i-1}^2}{1 - \bar{\alpha}_i^2}\beta_i^2.$$

$$\sum_{i=1}^L \mathbb{E}_{p(\mathbf{x}_i|\mathbf{x}_0)} [\mathcal{D}_{\text{KL}}(p(\mathbf{x}_{i-1}|\mathbf{x}_i, \mathbf{x}_0) \| p_{\phi}(\mathbf{x}_{i-1}|\mathbf{x}_i))].$$

$$\mathcal{L}_{\text{diffusion}}(\mathbf{x}_0; \phi) = \sum_{i=1}^L \frac{1}{2\sigma^2(i)} \|\boldsymbol{\mu}_{\phi}(\mathbf{x}_i, i) - \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}_0, i)\|_2^2 + C,$$



$$\mathcal{L}_{\text{diffusion}}(\mathbf{x}_0; \phi) = \sum_{i=1}^L \frac{1}{2\sigma^2(i)} \|\boldsymbol{\mu}_{\phi}(\mathbf{x}_i, i) - \boldsymbol{\mu}(\mathbf{x}_i, \mathbf{x}_0, i)\|_2^2 + C,$$

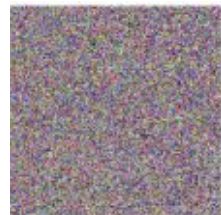
```
x_0 = data[idx]
```

```
# 1. Sample random timesteps t for each data point in the batch  
t = torch.randint(0, T, (batch_size,))
```

```
# 2. Sample random pure Gaussian noise epsilon  
noise = torch.randn_like(x_0)
```

```
# 3. Forward diffusion: Calculate x_t using the closed-form formula  
# x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * noise  
sqrt_alpha_bar_t = sqrt_alphas_cumprod[t].unsqueeze(-1)  
sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alphas_cumprod[t].unsqueeze(-1)  
  
x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise
```

```
# 4. Predict the noise that was added  
predicted_noise = model(x_t, t)  
# 5. Calculate loss and optimize  
loss = loss_fn(predicted_noise, noise)
```



```
# 1. Start with pure noise  $x_T \sim N(0, I)$ 
x = torch.randn((num_samples, 2))

# 2. Step backwards from T-1 down to 0
for i in reversed(range(T)):
    t = torch.full((num_samples,), i, dtype=torch.long)

    # Predict the noise using the model
    predicted_noise = model(x, t)

    # Fetch the schedule parameters for the current step t
    alpha = alphas[t].unsqueeze(-1)
    alpha_bar = alphas_cumprod[t].unsqueeze(-1)
    beta = betas[t].unsqueeze(-1)
```

```
# Reverse Step Formula:
```

```
#  $x_{t-1} = 1/\sqrt{\alpha} * (x_t - ((1-\alpha) / \sqrt{1-\alpha_{\text{bar}}}) * \text{predicted\_noise}) + \sqrt{\beta} * z$ 
x = (1.0 / torch.sqrt(alpha)) * (x - ((1.0 - alpha) / torch.sqrt(1.0 - alpha_bar)) * predicted_noise) + torch.sqrt(beta) * z
```

Score-based Generative Models

$$p(\mathbf{x}) = \frac{\tilde{p}(\mathbf{x})}{Z}, \quad Z = \int \tilde{p}(\mathbf{x}) \, d\mathbf{x}.$$

While computing Z is intractable, the score depends only on $\tilde{p}$:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z}_{=0} = \nabla_{\mathbf{x}} \log \tilde{p}(\mathbf{x}),$$



It is possible to sample from p only using the score function !

- Sample from $p(\mathbf{x})$ using only the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$
- Initialize $\mathbf{x}^0 \sim \pi(\mathbf{x})$
- Repeat for $t \leftarrow 1, 2, \dots, T$

$$\mathbf{z}^t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathbf{x}^t \leftarrow \mathbf{x}^{t-1} + \frac{\epsilon}{2} \nabla_{\mathbf{x}} \log p(\mathbf{x}^{t-1}) + \sqrt{\epsilon} \mathbf{z}^t$$

Stochasticity

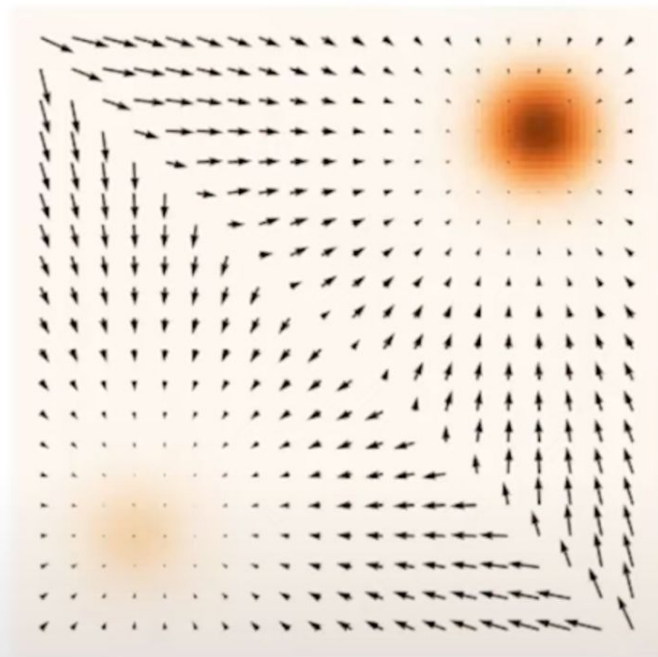
- If $\epsilon \rightarrow 0$ and $T \rightarrow \infty$, we are guaranteed to have $\mathbf{x}^T \sim p(\mathbf{x})$
- Langevin dynamics + score estimation

$$\mathbf{s}_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

$p(\mathbf{x})$
Probability density function



$\nabla_{\mathbf{x}} \log p(\mathbf{x})$
(Stein) score function



Score vs. density function

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_{\theta}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2]$$

(Fisher divergence)

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_{\theta}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2]$$

(Fisher divergence)

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of perturbed datapoints $\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_n\} \sim q_{\sigma}(\tilde{\mathbf{x}})$
 $\tilde{\mathbf{x}}_i \sim q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)$

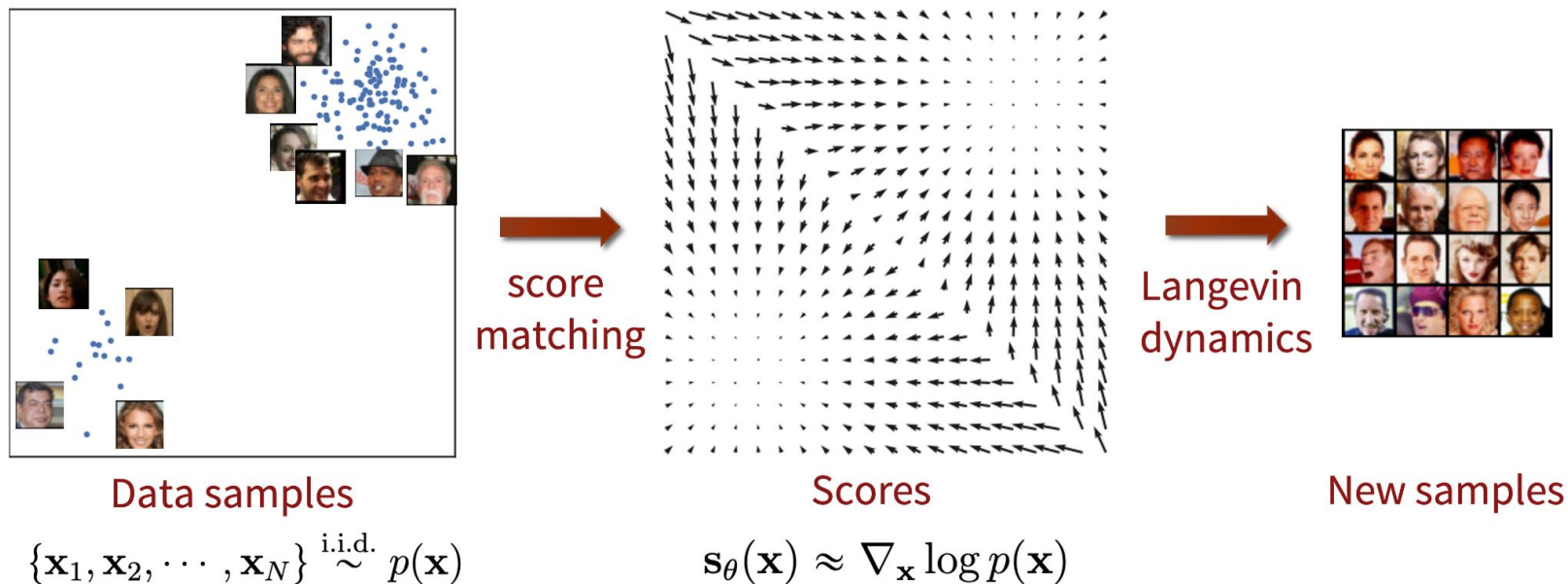
- Estimate the denoising score matching loss with empirical means

$$\frac{1}{2n} \sum_{i=1}^n [\|s_{\theta}(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)\|_2^2]$$

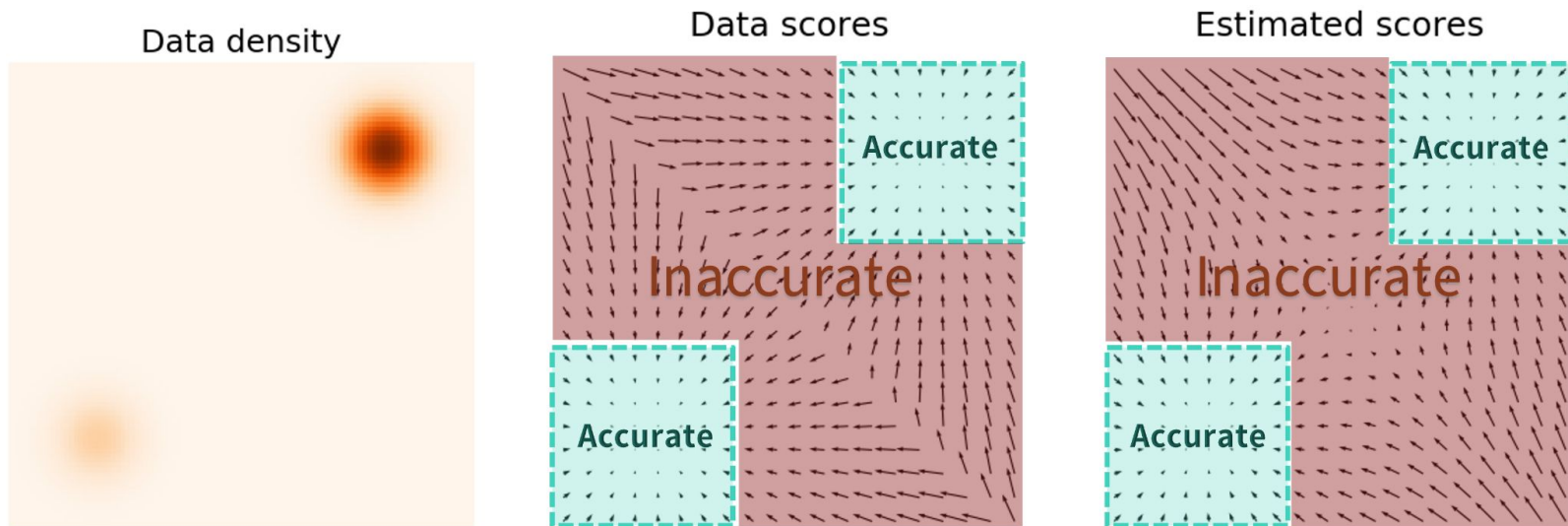
- If Gaussian perturbation

$$\frac{1}{2n} \sum_{i=1}^n \left[\|s_{\theta}(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2}\|_2^2 \right]$$

- Stochastic gradient descent
- Need to choose a very small σ !

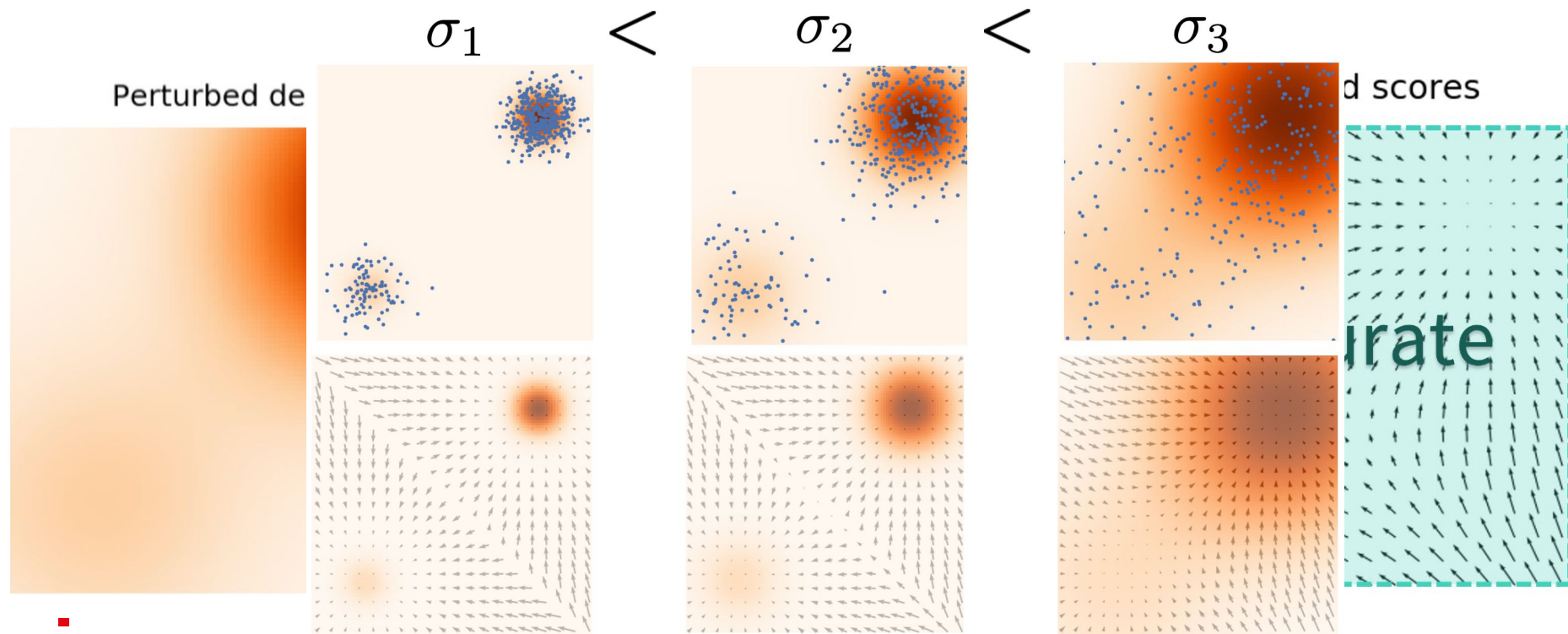


Any pitfalls ?



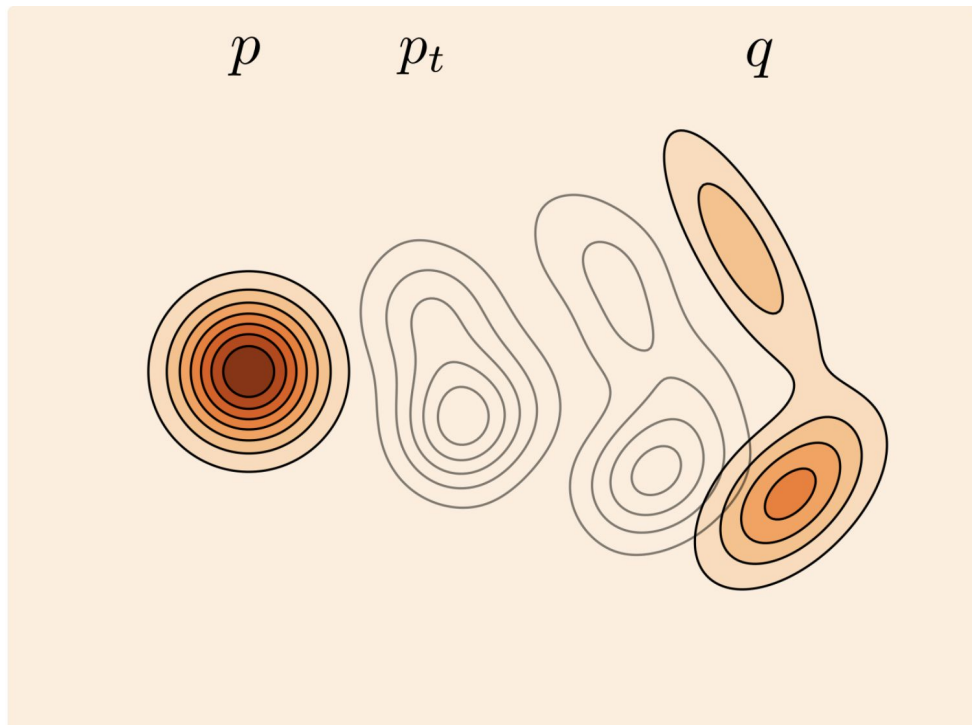
Inaccurate Estimation in low-density regions

A solution is to perturb the data points with different degree of noise and train score-based models on the noisy data points instead.

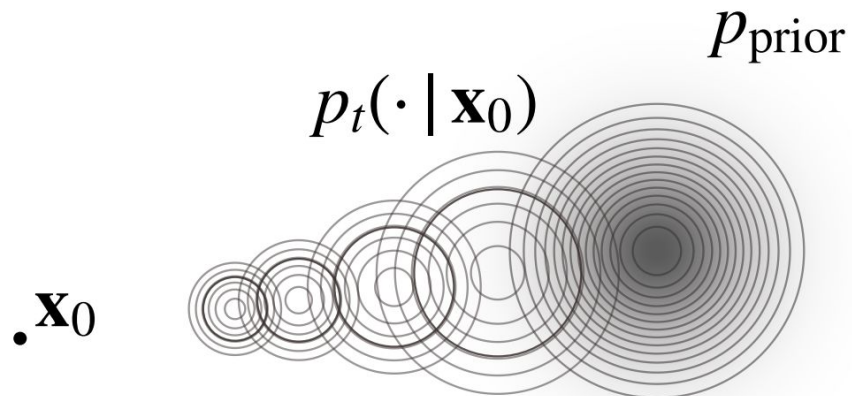


Flow Matching

- **Main Idea:** learn a continuous flow to transport samples from one distribution to the other.



- Conditional Path



- **Conditional Path**
- **Velocity Field:** find a velocity field that the induced ODE produces marginal distribution of $x(t)$ that matches with p_t at each time t .

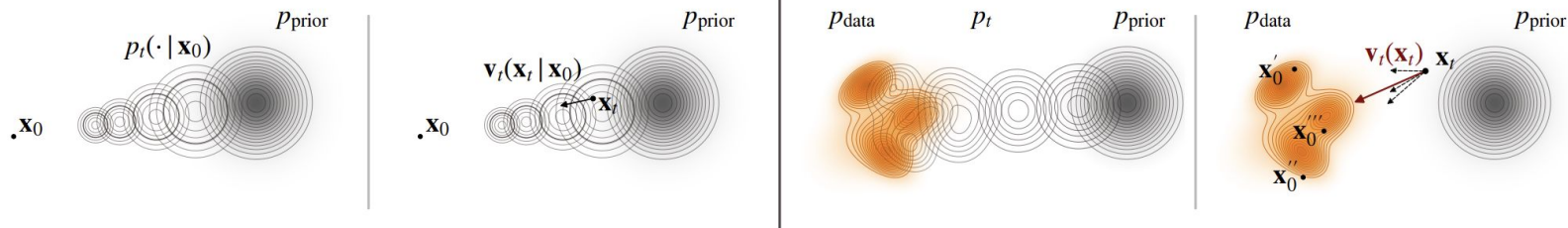
$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_t(\mathbf{x}(t)), \quad t \in [0, 1],$$

$$\frac{\partial p_t(\mathbf{x})}{\partial t} + \nabla \cdot (\mathbf{v}_t(\mathbf{x}) p_t(\mathbf{x})) = 0.$$

- **Conditional Path**
- **Velocity Field**
- **Learning via Conditional Strategy**

$$\mathcal{L}_{\text{FM}}(\phi) = \underbrace{\mathbb{E}_{t, \mathbf{z} \sim \pi(\mathbf{z}), \mathbf{x}_t \sim p_t(\cdot | \mathbf{z})} \left[\|\mathbf{v}_\phi(\mathbf{x}_t, t) - \mathbf{v}_t(\mathbf{x}_t | \mathbf{z})\|^2 \right]}_{\mathcal{L}_{\text{CFM}}(\phi)} + C.$$

- **Conditional Path**
- **Velocity Field**
- **Learning via Conditional Strategy**



Attention and Transformer

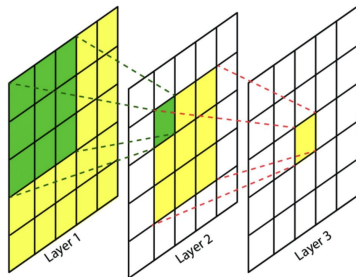
Why Transformers: Going Beyond CNN Models

□ Limitations of CNNs:

- Spatially local processing
- With increased depth: Lower resolution, Wider (richer) features

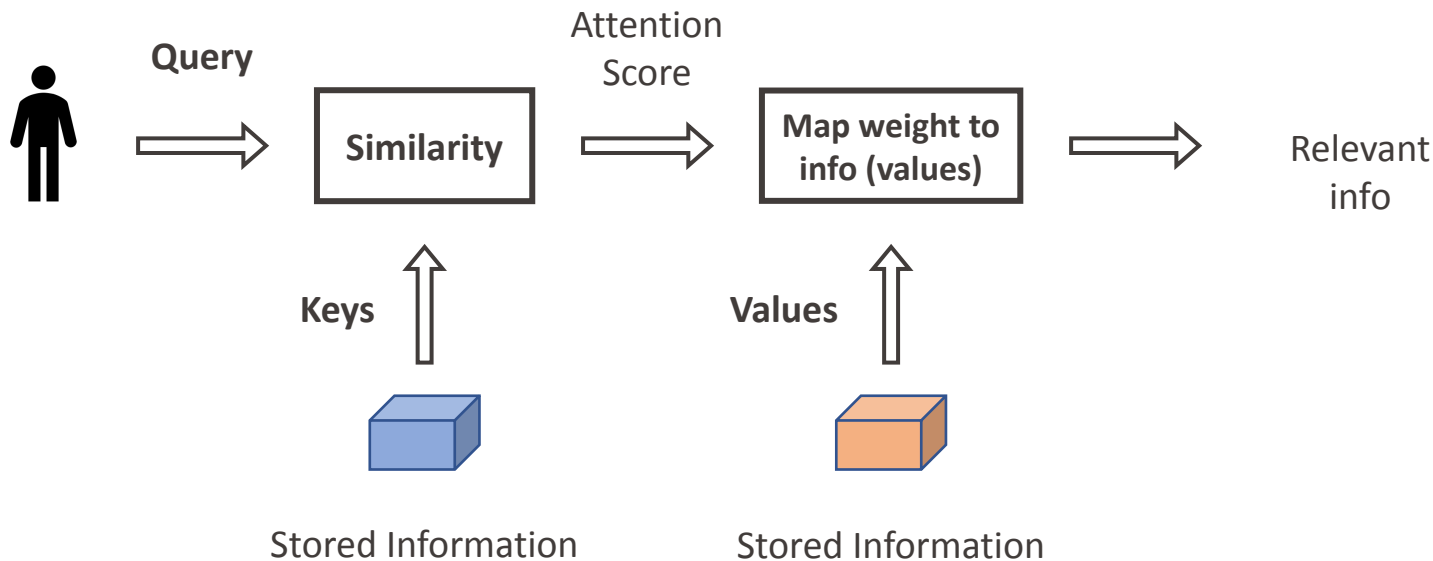
□ Desired Capabilities:

- Long range processing

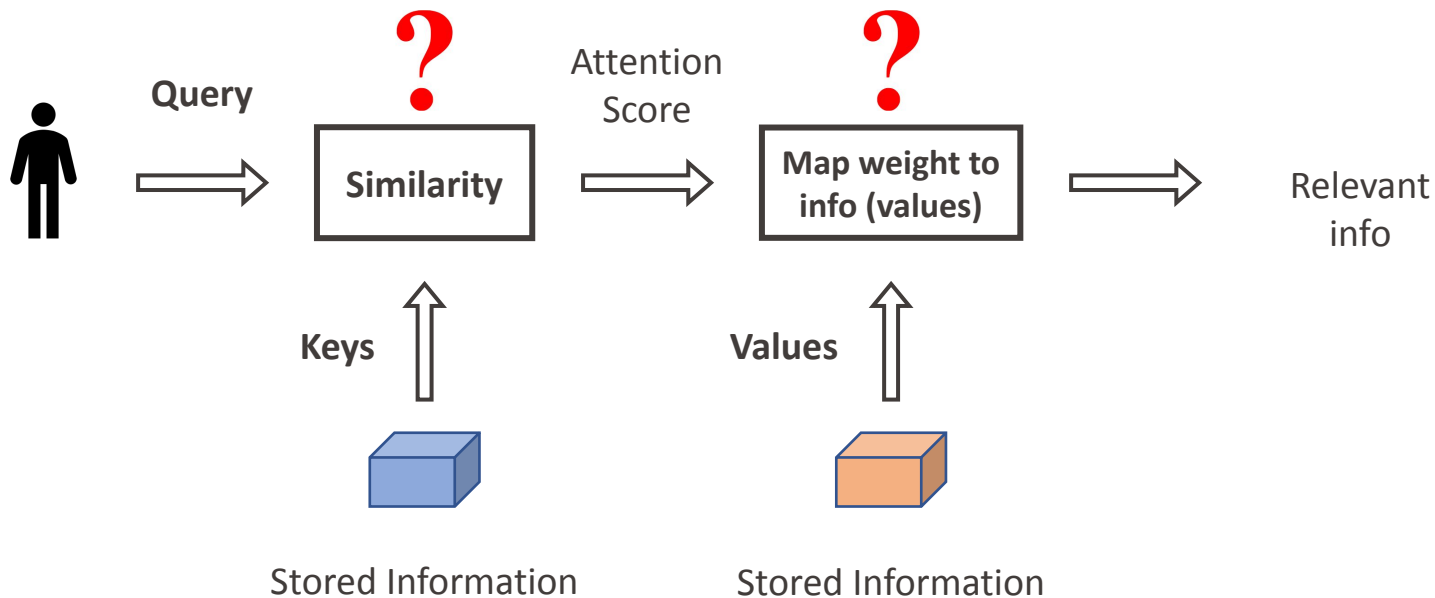


Understanding Attention

She is **eating** a **green** **apple**.

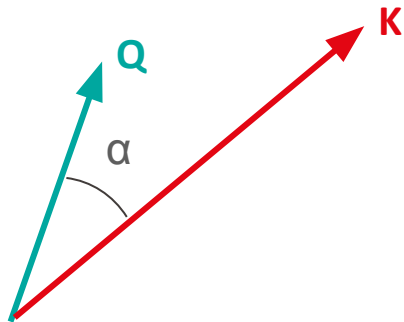


Understanding Attention



Attention Score

Compute pairwise similarity between each **query** and **key**.

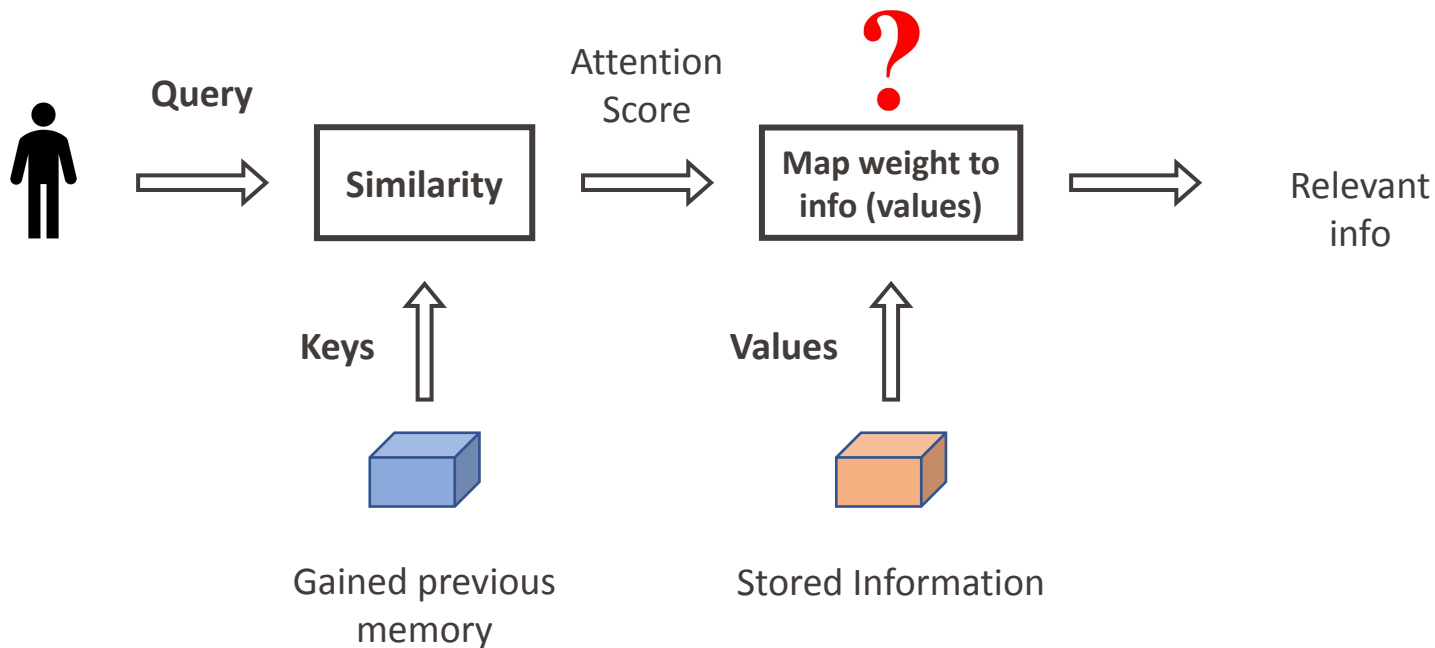


Dot product
→
Scale

$$\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\text{Scaling}}$$

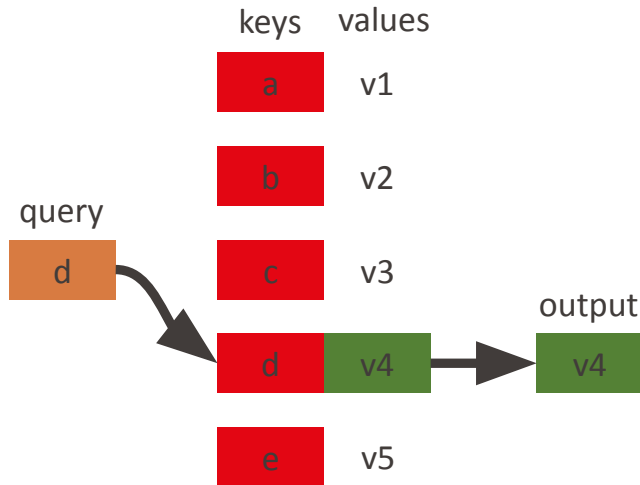
cosine similarity

Understanding Attention

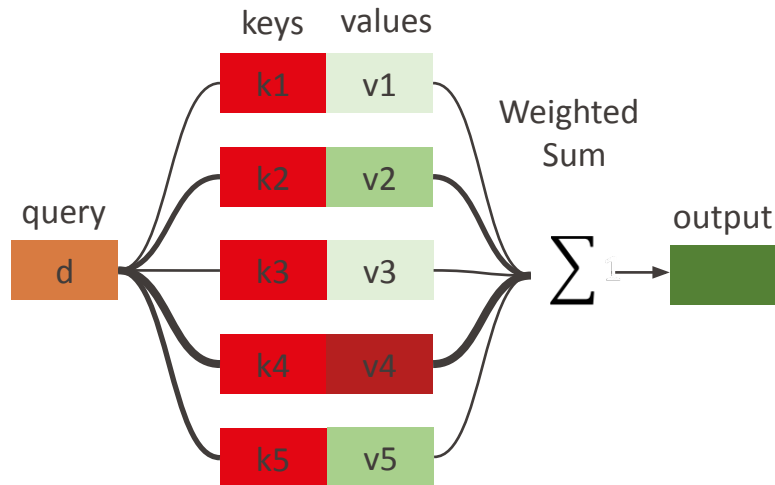


Understanding Attention

For a specific **query**, we have a table of **keys** that map to **values**. The **query** matches one of the **keys**, returning its **value**.



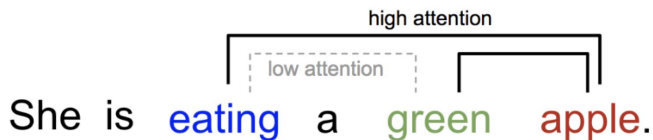
The **query** matches all **keys** softly, to a weight between 0 and 1. The **keys'** **values** are multiplied by the weights and summed.



$$\text{output} = \sum_{i=1}^5 \alpha_i V_i \quad ; \quad \alpha_i \text{ are attention weights}$$

Self-attention

Attention: A mechanism to selectively exchange and process information



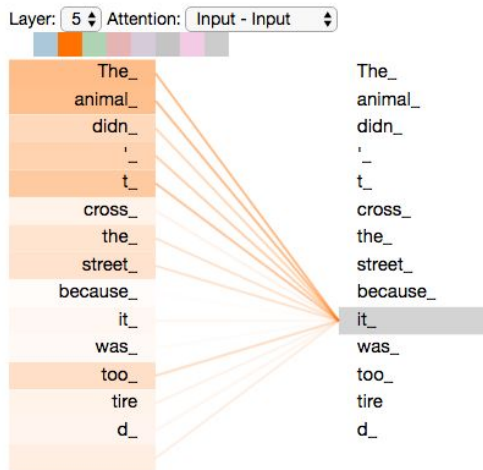
How? By averaging **values** associated to **keys** matching a **query**

Self-attention: Attention from each token to all other tokens

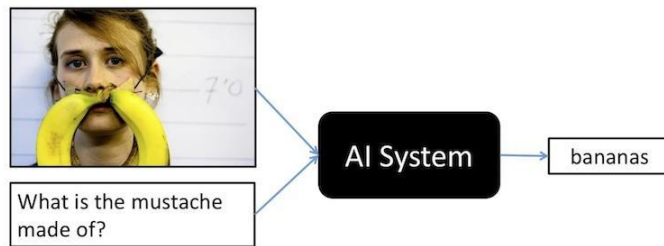
[Attention? Attention! \(Lilian Weng, 2018\)](#)

Self-Attention vs Cross-Attention

- Self-Attention: Queries, keys, and values are generated from the same source.



- Cross-Attention: Queries are generated from a source different from keys, and values.



[VQA: Visual Question Answering, ICCV 2015](#)

[The Illustrated Transformer \(Jay Alammar, 2018\)](#)

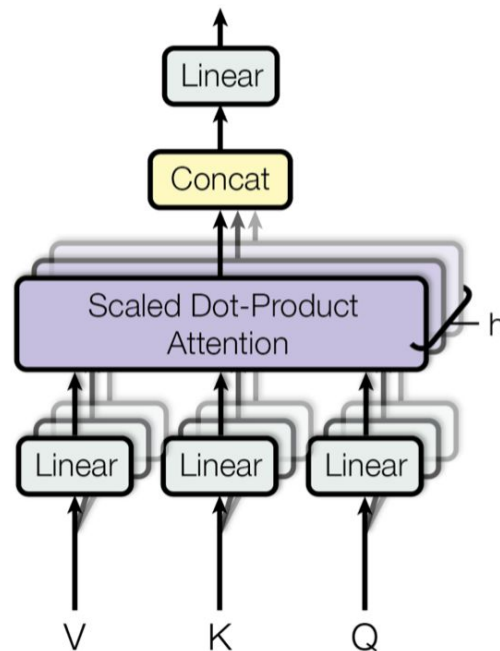
Multi-head attention

Idea: Perform self-attention multiple times in parallel and combine the results!

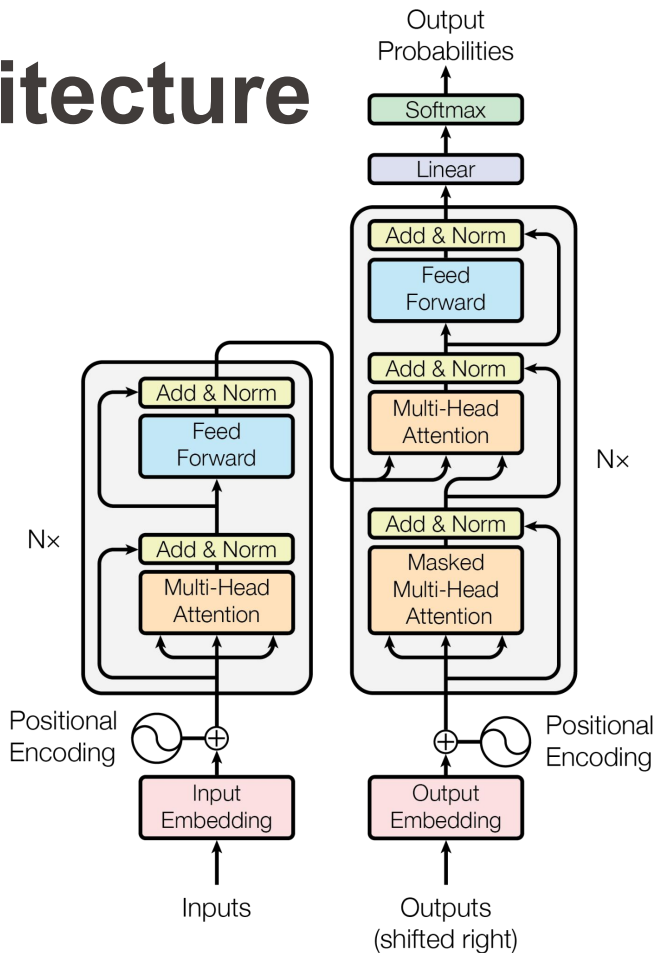
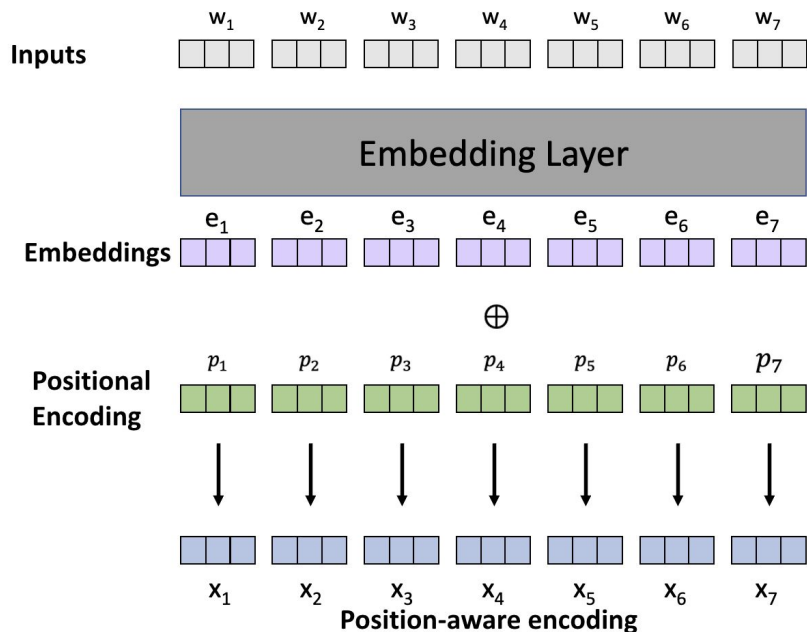
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$

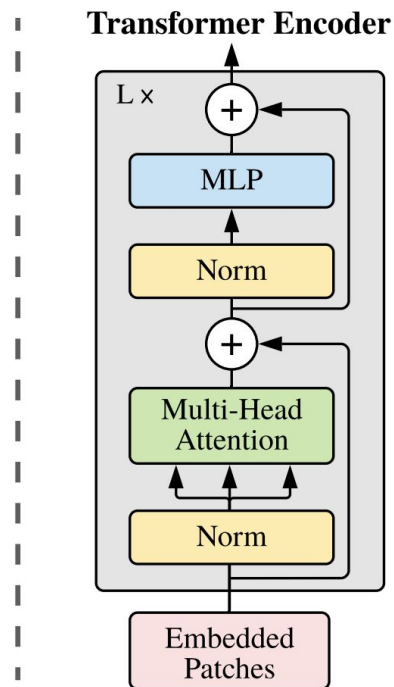
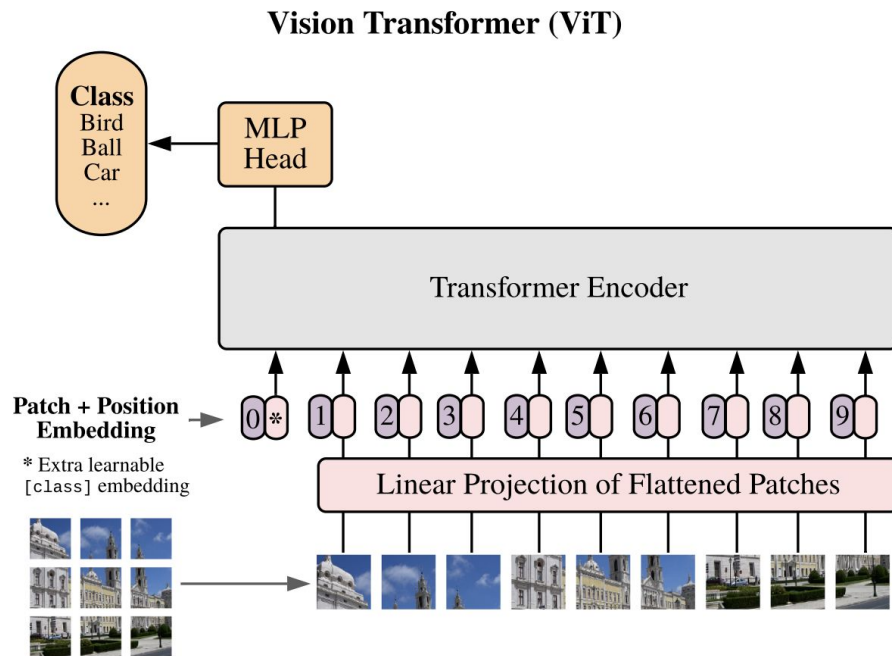
Still easy to vectorize, but the model can now jointly attend to information from different subspaces at different positions



The Transformer architecture

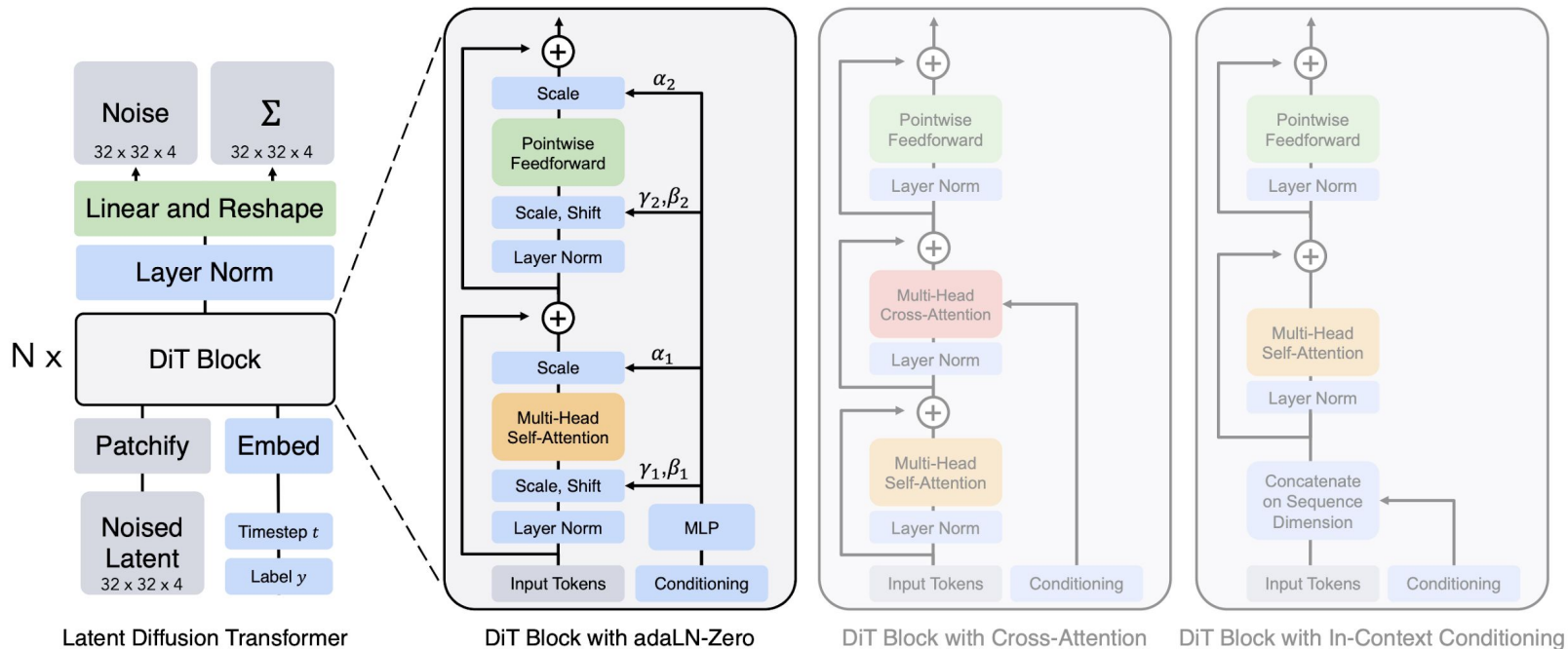


Vision Transformer

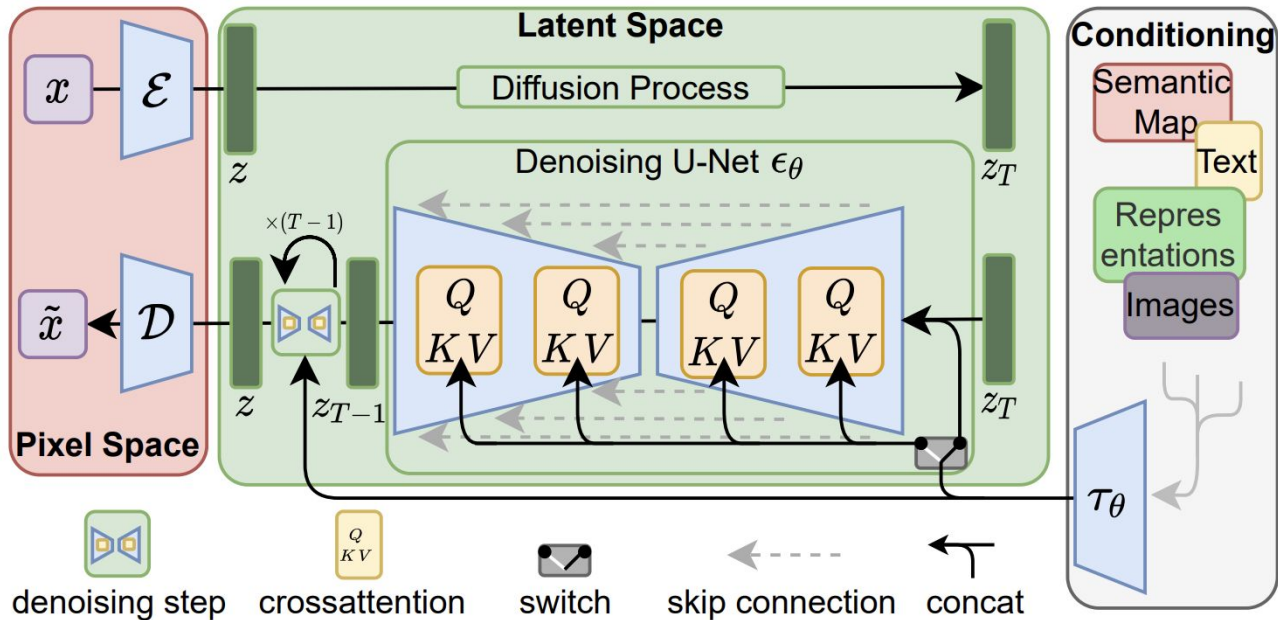


■ AN IMAGE IS WORTH 16X16 WORDS

Diffusion Transformer



Latent Diffusion model



Controllable Generation

$$\epsilon_{\theta}(x_t, t, c), \quad x_{\theta}(x_t, t, c), \quad s_{\theta}(x_t, t, c), \quad v_{\theta}(x_t, t, c)$$

- First, when training the model, we add the condition on top of the temporal condition.

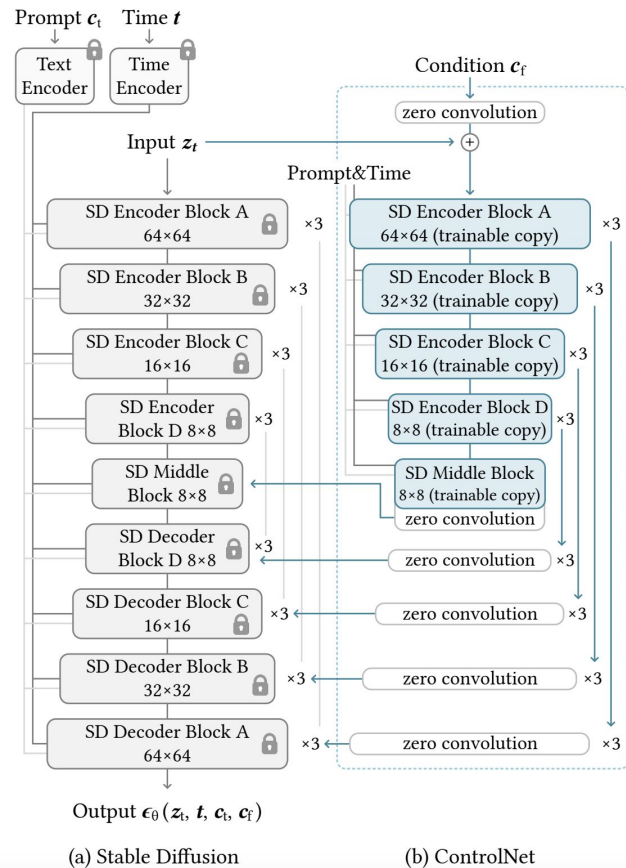
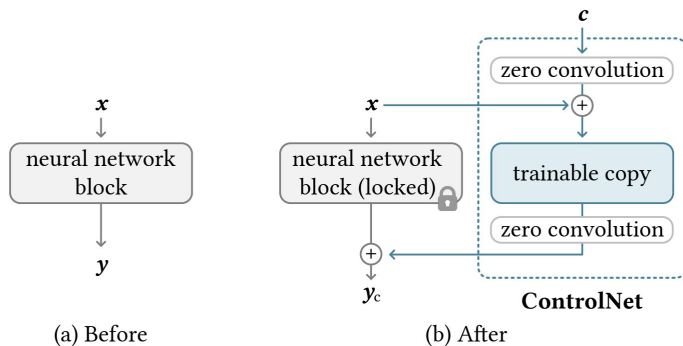
```
predicted_noise = model(x_t, t, c)
```

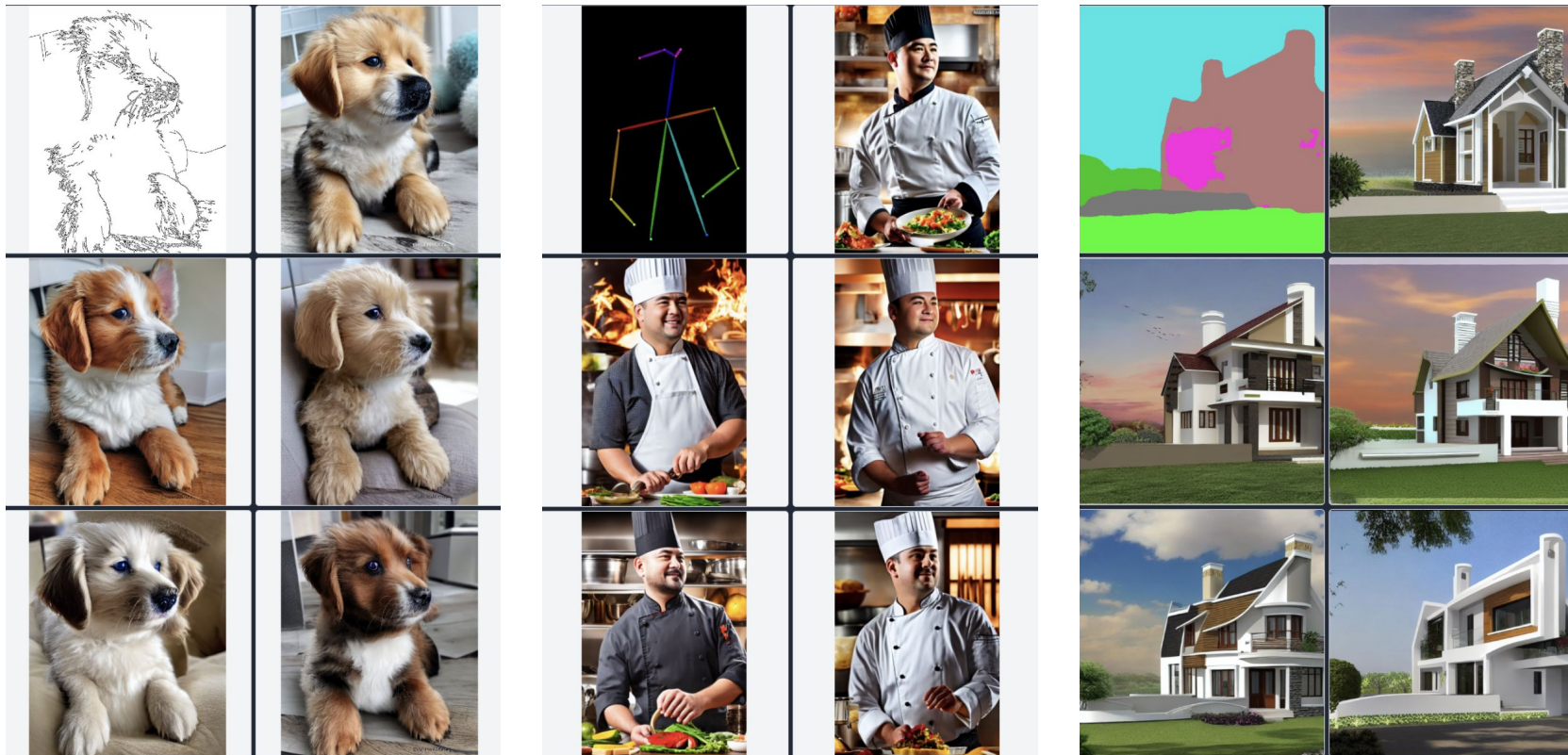


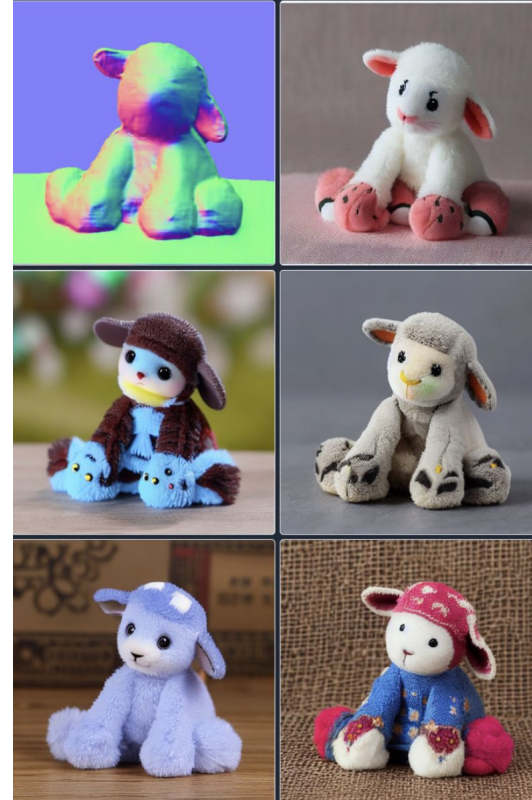
$$w \cdot \epsilon_{\theta}(x_t, t, c) + (1 - w) \cdot \epsilon_{\theta}(x_t, t, \emptyset),$$

where w is the guidance strength.





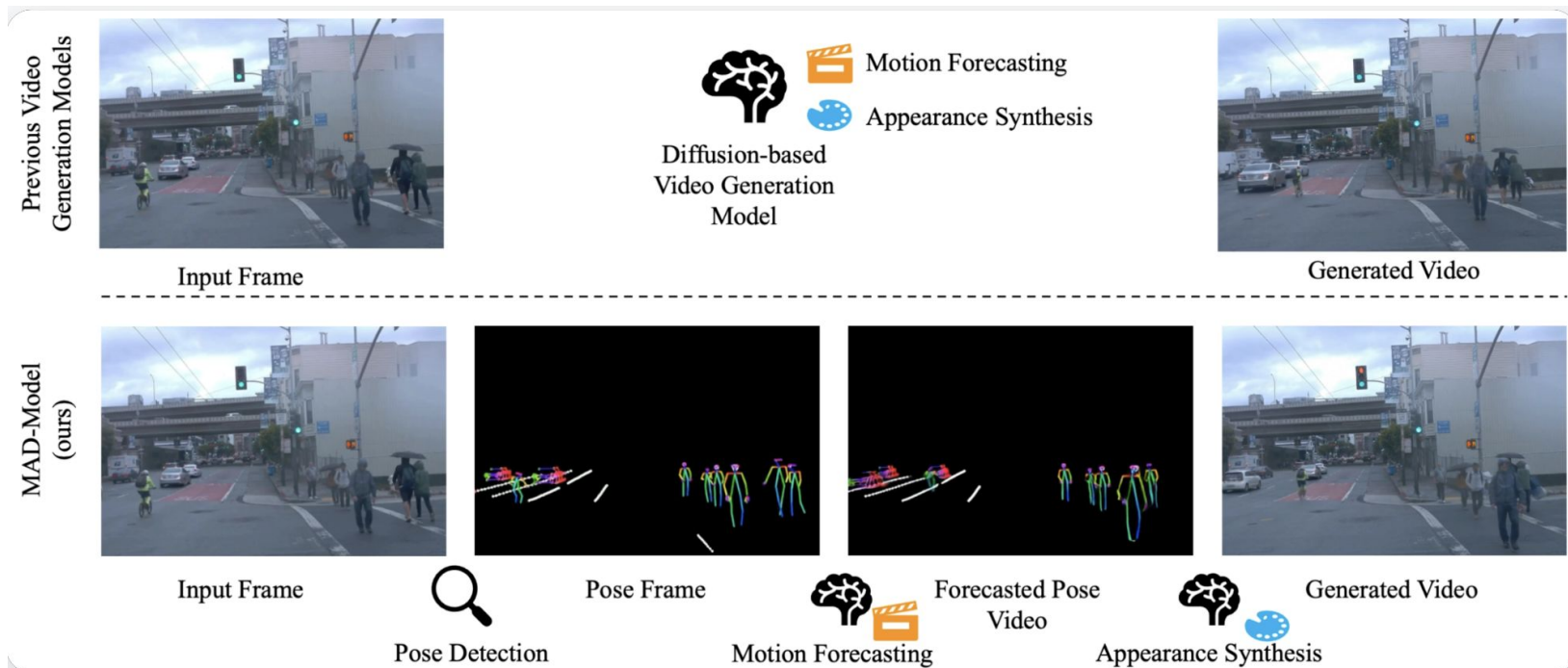




World Modeling



■ **GEM: A Generalizable Ego-Vision Multimodal World Model for Fine-Grained Ego-Motion, Object Dynamics, and Scene Composition Control**



MAD: Motion Appearance Decoupling for efficient Driving World Models

Accelerating the Training

Motivation

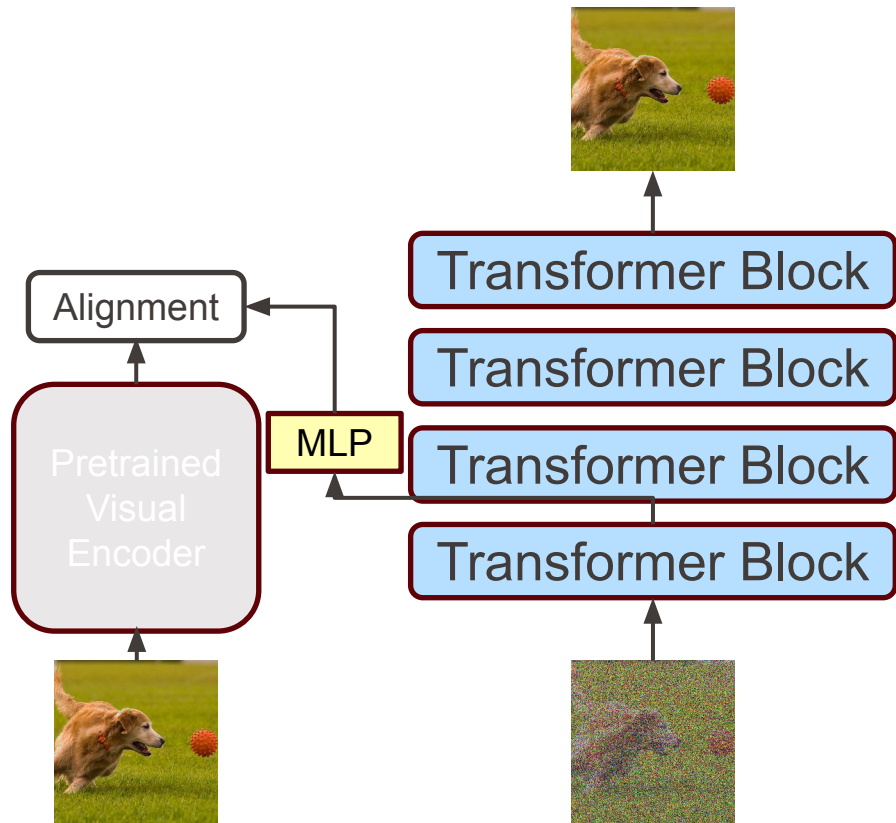


NVIDIA

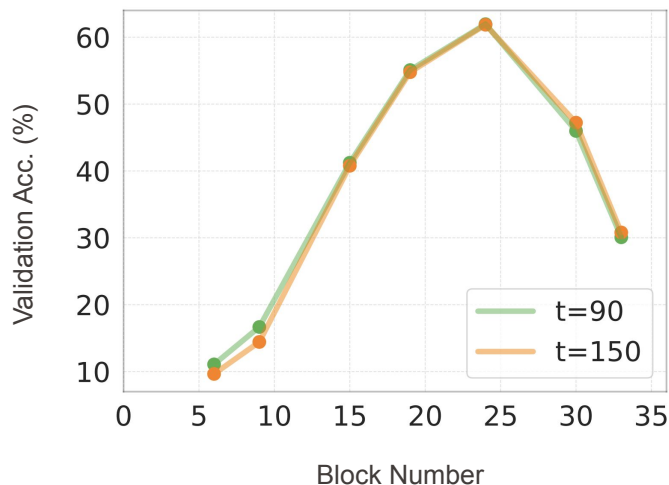
Cosmos was trained on **10,000 GPUs** over **3 months**.



Imagen 3 was trained on 1024 **TPUs** for **several months**.



- Accelerates training
- Extra cost and parameters
- Not easy to extend beyond visual domain



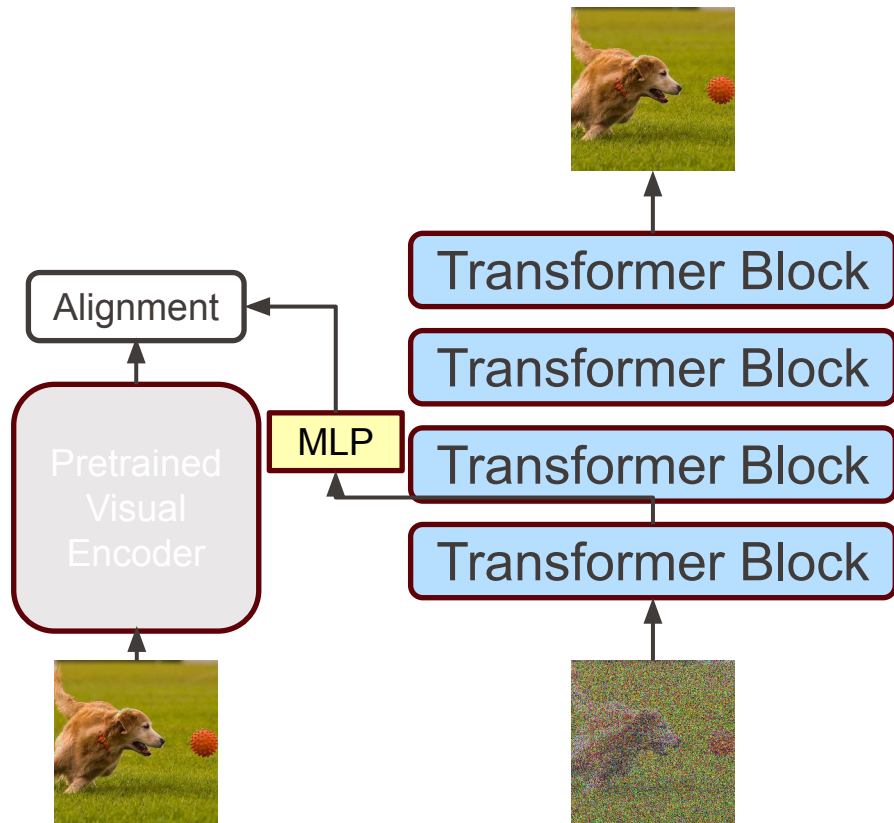
Linear probing of diffusion models for classification on ImageNet (256×256) shows that the deeper layers learn discriminative features.

Xiang, Weilai, et al. "Denoising diffusion autoencoders are unified self-supervised learners." ICCV 2023.

Mukhopadhyay, Soumik, et al. "Do text-free diffusion models learn discriminative visual representations?." ECCV 2024.

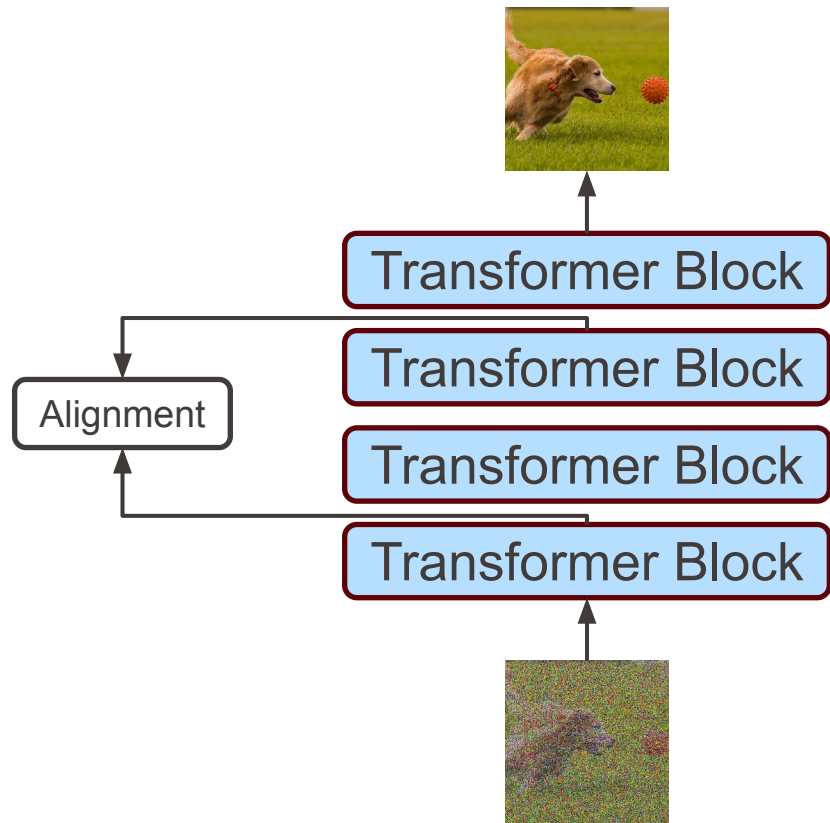
Stracke, Nick, et al. "Cleadift: Diffusion features without noise." CVPR 2025.

REPA

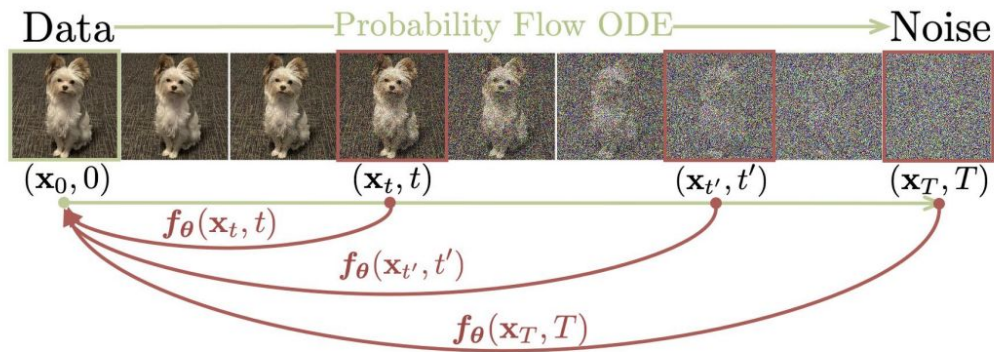


LayerSync

70



Accelerating the Inference



Input: dataset $\mathcal{D}$, initial model parameter θ , learning rate η , step schedule $N(\cdot)$, EMA decay rate schedule $\mu(\cdot)$, $d(\cdot, \cdot)$, and $\lambda(\cdot)$
 $\theta^- \leftarrow \theta$ and $k \leftarrow 0$

repeat

Sample $\mathbf{x} \sim \mathcal{D}$, and $n \sim \mathcal{U}[1, N(k) - 1]$

Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$\mathcal{L}(\theta, \theta^-) \leftarrow$

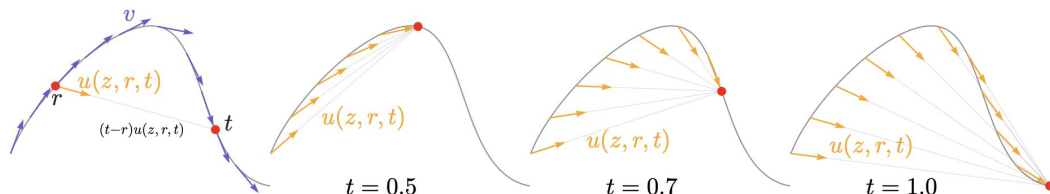
$\lambda(t_n) d(\mathbf{f}_{\theta}(\mathbf{x} + t_{n+1}\mathbf{z}, t_{n+1}), \mathbf{f}_{\theta^-}(\mathbf{x} + t_n\mathbf{z}, t_n))$

$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta, \theta^-)$

$\theta^- \leftarrow \text{stopgrad}(\mu(k)\theta^- + (1 - \mu(k))\theta)$

$k \leftarrow k + 1$

until convergence

**Algorithm 1** MeanFlow: Training.

Note: in PyTorch and JAX, jvp returns the function output and JVP.

```
# fn(z, r, t): function to predict u
# x: training batch
```

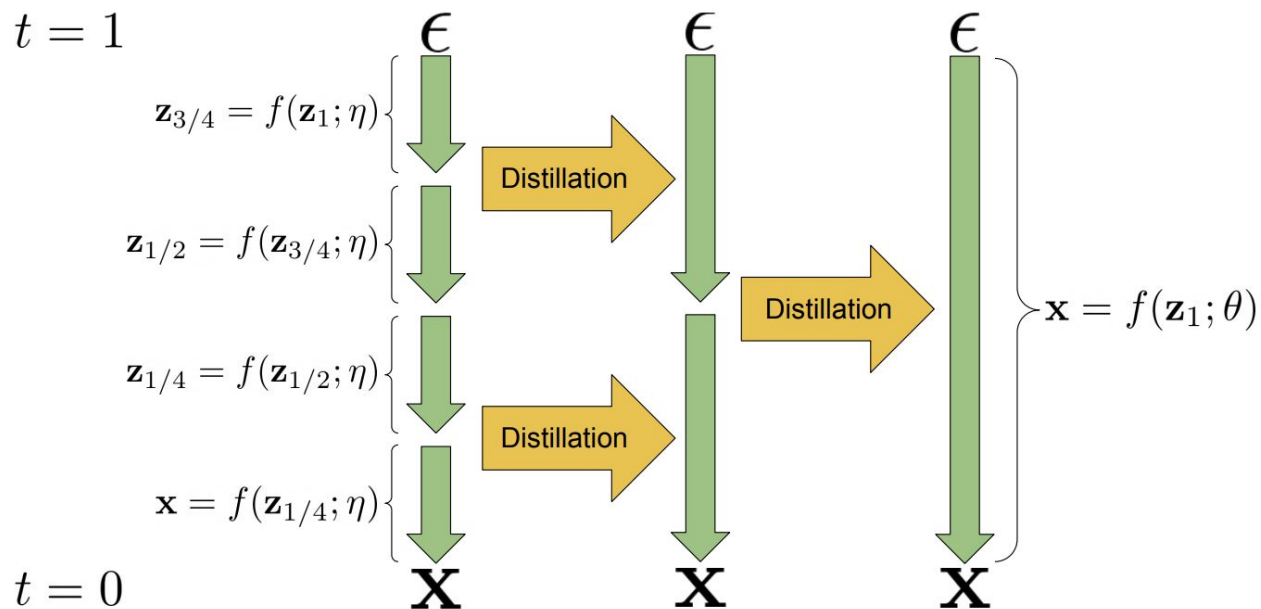
```
t, r = sample_t_r()
e = randn_like(x)
```

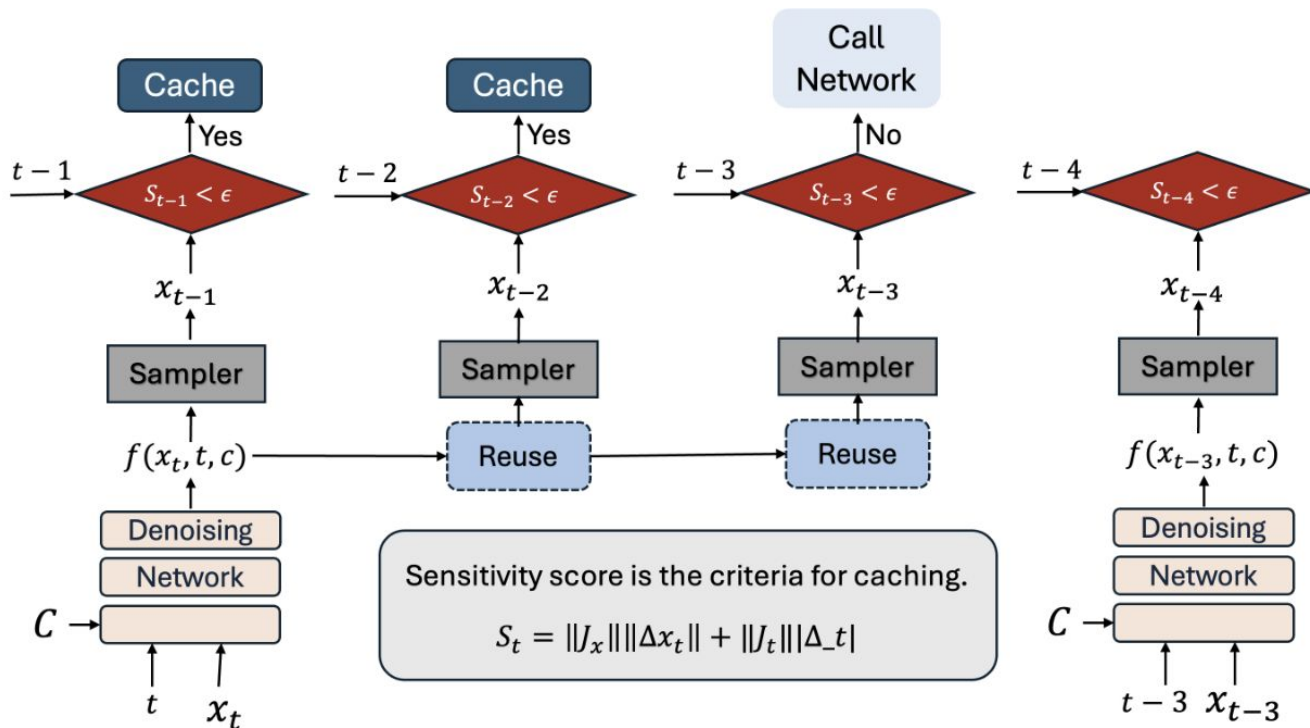
```
z = (1 - t) * x + t * e
v = e - x
```

```
u, dudt = jvp(fn, (z, r, t), (v, 0, 1))
```

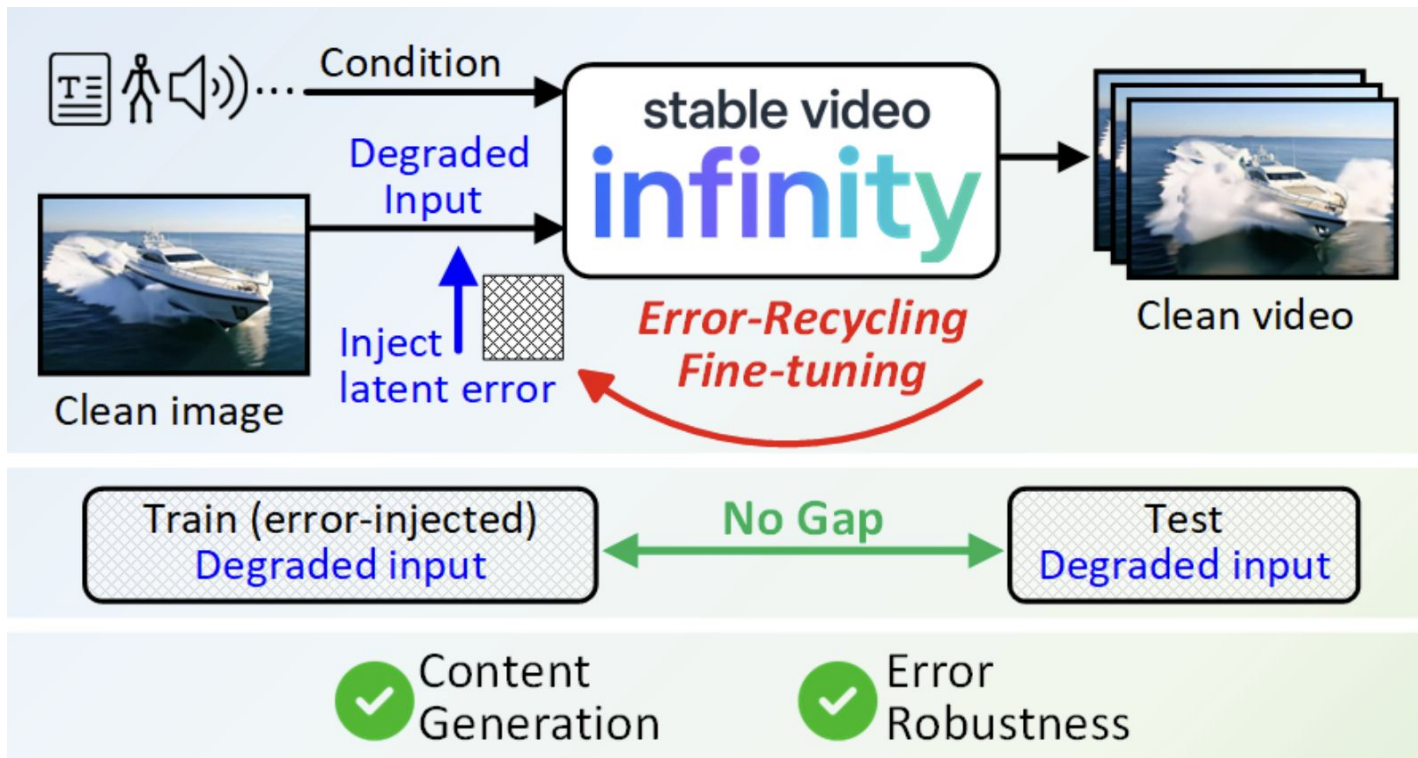
```
u_tgt = v - (t - r) * dudt
error = u - stopgrad(u_tgt)
```

```
loss = metric(error)
```





Long Generation



Thank you for your attention